

ENHANCING SMISHING DETECTION: A DEEP LEARNING APPROACH FOR IMPROVED ACCURACY

A thesis submitted in partial fulfillment of the requirements for
the award of the degree of

B. Tech.

in

Computer Science and Engineering

By

Sambit Kumar Bisoi (Reg No.2201204084)

Subham Gouda (Reg No. 2201204097)

R Amrit Raj (Reg No. 2201204069)

Subrat Gouda (Reg No.2201204101)



**COMPUTER SCIENCE AND ENGINEERING
ROLAND INSTITUTE OF TECHNOLOGY GOLONTHARA OD 761008**

April 2026

In memory of

Late Dr J Surya Rao “Founder Chairman” RIT Our deepest gratitude.

BONAFIDE CERTIFICATE

This is to certify that the project titled **ENHANCING SMISHING DETECTION: A DEEP LEARNING APPROACH FOR IMPROVED ACCURACY** is a bonafide record of the work

done by

Sambit Kumar Bisoi (Reg No. 2201204084)

Subham Gouda (Reg No. 2201204097)

R Amrit Raj (Reg No. 2201204069)

Subrat Gouda (Reg No. 2201204101)

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** of the **ROLAND INSTITUTE OF TECHNOLOGY, GOLONTHARA, OD 761008**, during the year 2025-26.

Prof. Sanjaya Kumar Routa
Professor in Dept. of CSE
Project Guide

Prof. (Dr.) Gunamani Jena
Principal, RIT

Prof. (Dr.) Sunil Kumar Sahu
Head of the Department of CSE

Project Viva-voce held on _____

Internal Examiner

External Examiner

ABSTRACT

The rapid increase in mobile communication has led to a rise in **smishing (SMS phishing) attacks**, which pose significant risks to user security and privacy. Traditional detection methods, including rule-based systems and conventional machine learning models, often fail to accurately identify such attacks due to their limited ability to understand contextual and sequential patterns in text. As a result, these methods frequently produce high false positive rates, misclassifying legitimate messages as malicious.

To overcome these limitations, this project proposes an advanced **ensemble deep learning model** that combines Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Bidirectional LSTM (BiLSTM) architectures. The model utilizes **character-level tokenization** to capture fine-grained textual features, including variations and hidden patterns commonly found in smishing messages. CNN is used for feature extraction, while LSTM and BiLSTM layers help in understanding the sequential and contextual relationships within the text, improving the overall classification performance.

Experimental results demonstrate that the proposed model achieves higher accuracy and significantly reduces false positives compared to traditional approaches. By effectively integrating multiple deep learning techniques, the system provides a reliable and scalable solution for smishing detection. This work contributes to enhancing cybersecurity measures and can be applied in real-world systems such as SMS filtering and mobile security applications.

ACKNOWLEDGEMENTS

We would like to express our deepest gratitude to the following people for guiding us through this course and without whom this project and the results achieved from it would not have reached completion.

Prof. Sanjaya Kumar Routa, Assistant Professor in Department of Computer Science and Engineering, for helping us and guiding us in the course of this project. Without his guidance, we would not have been able to successfully complete this project. His patience and genial attitude is and always will be a source of inspiration to us.

Prof. (Dr.) Sunil Kumar Sahu, the Head of the Department, Department of Computer Science and Engineering, for allowing us to avail the facilities at the department.

We are thankful to the Principal **Dr Gunamani Jena** for providing the facility in this institution.

We are also thankful to the Honorable Chairperson Mrs J Jayalakshmi for motivating, arranging training and internship related to the Project.

We are also thankful to the faculty and staff members of the Department of Computer Science and Engineering, our individual parents and our friends for their constant support and help.

TABLE OF CONTENTS

Title	Page No.
ABSTRACT	i
ACKNOWLEDGEMENTS	ii
TABLE OF CONTENTS	iii
LIST OF FIGURES	v
LIST OF SCREENS	vi
CHAPTER 1 Introduction	1
1.1 How Machine Learning Works	2
1.1.1 Machine Learning Algorithms	2
1.1.2 Types of Machine Learning Methods.....	3
1.1.3 Applications of Machine Learning	4
1.1.4 What is Deep Learning?.....	5
1.1.5 How Deep Learning Works	6
CHAPTER 2 Literature Survey	8
CHAPTER 3 System Analysis	12
3.1 Existing System	12
3.2 Proposed System	13
CHAPTER 4 Fundamental Requirement	14
4.1 System Functional Requirement.....	15
CHAPTER 5 Non-Functional Requirement	17
5.1 Requirements.....	17
CHAPTER 6 Feasibility Study	19

CHAPTER 7 Requirement Specification	20
7.1 Hardware Requirements.....	20
7.2 Software Requirements.....	20
CHAPTER 8 System Design.....	21
8.1 Methodology	21
8.2 Dataset	23
8.3 System Architecture.....	23
CHAPTER 9 UML Design	24
9.1 Use Case Diagram.....	25
9.2 Class Diagram.....	26
9.3 Sequence Diagram	26
9.4 Collaboration Diagram	27
9.5 Activity Diagram.....	28
CHAPTER 10 Technology Descriptions.....	30
10.1 Python History	31
10.2 Python Features	31
10.3 Python Applications	33
10.4 Why Python	34
10.5 Python Installation.....	35
CHAPTER 11 Sample Code	39
CHAPTER 12 Testing.....	42
12.1 Introduction.....	42
12.1.1 System Testing and Implementation.....	42
12.1.2 Testing Techniques	43
12.1.3 Testing Strategies.....	43

CHAPTER 13 Screenshots.....	45
CHAPTER 14 Conclusion.....	51
CHAPTER 15 Bibliography	52

LIST OF FIGURES

Figure 1 Architecture	23
Figure 2 Use case Diagram	25
Figure 3 Class Diagram	26
Figure 4 Sequence Diagram	27
Figure 5 Collaboration Diagram	28
Figure 6 Activity Diagram	29

LIST OF SCREENS

Screen 1 Dataset Class Label Graph	45
Screen 2 BI-LSTM	45
Screen 3 GRU Algorithm.....	46
Screen 4 LSTM Algorithms	46
Screen 5 CNN + LSTM	47
Screen 6 CNN + LSTM + Bidirectional	47
Screen 7 All Algorithms Performance Graph	48
Screen 8 Home Page	48
Screen 9 Admin Login.....	49
Screen 10 Admin Home.....	49
Screen 11 Upload 'testData.csv' file	50
Screen 12 SMS Smishing Detection.....	50

CHAPTER 1

Introduction

In today's digital age, smartphones have become an essential part of our daily lives. With easy access to the internet and countless mobile applications, people now rely heavily on their phones for everything from banking and shopping to communication and entertainment. However, this widespread use of mobile devices has also made them a prime target for cybercriminals. One common method attackers use to exploit smartphone users is through **smishing**, a form of phishing carried out via SMS (Short Message Service).

Smishing, which combines the words "SMS" and "phishing," refers to malicious text messages designed to trick people into revealing personal or sensitive information such as passwords, credit card details, or login credentials. These messages often appear to be from trusted organizations like banks, delivery companies, or government agencies, urging recipients to click on suspicious links or reply with confidential information. Because SMS messages are typically short, immediate, and viewed quickly, they often succeed in deceiving users—making smishing a significant and growing cybersecurity threat.

The COVID-19 pandemic further increased digital communication, and with it came a rise in smishing attacks. Fraudsters took advantage of the global crisis by sending messages pretending to be from health authorities or financial support services. These smishing attempts directed users to malicious websites or encouraged them to download harmful applications. The urgency and fear surrounding the pandemic made people more vulnerable to such attacks. The United Nations even reported a **350% increase in phishing attacks**, including smishing, during the pandemic period.

What makes smishing especially dangerous is the **difficulty in detecting it**. Unlike email, which often has spam filters and security layers, SMS does not typically have advanced filtering mechanisms. Many users are unaware of the risks or fail to verify the authenticity of SMS content. Moreover, with limited screen size and simplified interfaces, smartphones make it harder for users

to spot red flags that would otherwise be easier to identify on larger devices.

Various solutions have been proposed to combat smishing, including blocklists (which block known malicious senders or links) and heuristic-based systems (which look for specific keywords or patterns in messages). However, these approaches often fall short. For example, blocklists become ineffective when attackers frequently change domain names or use URL shortening services. Heuristic methods may mistakenly flag legitimate messages as threats, leading to a high number of **false positives**—where a safe message is wrongly labeled as malicious. This not only reduces user trust but also disrupts normal communication.

1.1 How Machine Learning Works

To begin with, this and computer science and, more particularly, AI and its subset called machine learning focuses on the collection of data and making use of algorithms to accomplish the objective of replicating the human learning processes more accurately rather than attempting to entirely imitate them. This is critical for data science as it allows systems to be able to uncover complex relationships within data that can make predictions or classifications automatically, and later using these relationships to make decisions within a business or any other establishment. Such insights are essential, especially for data mining initiatives and can have critical impact on major business development indices.

However, as the volume of big data is increasing so does the need for data scientists who understand the business perspective and the problems that needs solving and how data can provide the answers to the problems. This is where machine learning becomes relevant as it provides an opportunity to solve all these problems by automating the process through the use of statistical methods in training algorithms to recognize underlying relationships or patterns with data.

1.1.1 Machine Learning Algorithms

There are three important principles on which machine learning algorithms evolve.

1. A Decision Process.

This is the formulization of the problem which has to be solved by the machine learning. The

problem solving begins with the identification of what the problem is and this means stating what are the objectives of the problem

2. An Error Function

The algorithm's predictions are ascertained with the aid of the error function. The error function is an evaluation tool, which verifies the efficiency of the model based on the output in relation to the available known situations and indicates places where improvements can be made in the model.

3. A Model Optimization Process

During the optimization process, the already defined model is employed to enhance its accuracy by modifying parameters, for instance, weights, that determine the alignment of the model with the data. The model conducts its own evaluation, causes some changes which are afterward subjected to further evaluation, and repeats everything until it fully satisfies itself with the accuracy of its prediction, or the process exceeds some limits preset.

1.1.2 Types of Machine Learning Methods

Supervised Machine Learning

Initiating with labelled data leads to supervised learning algorithms, thereby informing the input and the output. In this manner, the model is able to understand the rules and the relationships of the data so as to classify patterns and provide predictions of unseen data. Adding the data would energise the model to make weight adjustments to the parameters so as to better the performance during the training phase. Cross-validation is adopted for balancing cases where overfitting (not too general) and underfitting situations (too general) are avoided.

Unsupervised Machine Learning

On the contrary, unsupervised learning involves working with unlabelled sets of data. Such algorithms examine the input data to discover and cluster the data into groups or structures that were not taught to them. This makes it very useful for exploratory data analysis, for market segmentation, cross-marketing, or for recognizing images and patterns.

Semi-Supervised Learning

To do this, semi-supervised learning bridges supervised and unsupervised learning by employing a few labeled data and lots of unlabeled data. The final understanding of the academic efforts is achieved through feature extraction and image collection from the labeled files, hence the model is able to understand the unrefereed files. This model proves useful in situations where the labeled data is scarce or the processes of labeling data is expensive or infeasible.

1.1.3 Applications of Machine Learning

1. Voice Identification

Automatic Speech Recognition (ASR) also known as voice recognition is that aspect of natural language processing (NLP) wherein spoken words are converted to written texts. This technology is quite widely used on portable devices, for instance, mobile phones, where people are able to do voice searches on devices such as Siri and can also send text messages without using their hands.

2. Customer support

Customer interaction and engagement are being revolutionized by AI powered chatbots. Customers can be assisted on sites and across social media platforms. These AI powered bots can answer frequently asked questions about shipping conditions, give fitting and size suggestions, and recommend goods to buy, all of which help to maximize customer interaction. These can include ecommerce virtual agents, slack and facebook messenger bots, and voice interface systems.

3. Vision AI

With the help of computer vision, systems can analyze faces, videos and other visual aspects to gain an understanding of the situation at hand. Unlike simple image recognition, such approaches can view pictures and videos and consider them for decision making. These include:

- Tagging of pictures posted on social media sites.
- Healthcare radiology imaging.

- Automobiles with automated driving features in the automobile sector.

4. Virtual shopping assistants

Past behavioral data of users is outputted by an AI algorithm and then analyzed to find valuable insights which can be used to strategically target users. This is most common in e-commerce stores where clients are offered recommendations about items that can be purchased in addition to what the user initially intended to buy.

5. Stock trading robots

The new trading platforms that have been developed focus on algorithmic trading of stocks while an AI powered engine assembles the stock portfolio. Daily, these computers can place hundreds of thousands or millions of trades, making them suitable for stock markets.

1.1.4 WHAT IS DEEP LEARNING?

Deep learning is a subfield of AI that engages people's attention the most today. This is the case because deep learning allows the automation of tasks such as image classification, speech recognition, object detection, and content description. In short, deep learning allows machines to comprehend data in a more advanced way.

For example, deep learning is used in image classification, speech recognition, object detection, and content analysis. The exponential growth can be attributed to several factors:

Key Drivers of Deep Learning Advancements

1. Algorithmic Improvements

The innovations in the algorithms have drastically increased the usability of the deep learning models.

2. New Machine Learning Approaches

The methodologies have evolved over the years, and that has retained the prediction accuracy of the models as well as the decisions made.

3. Specialized Neural Networks

New formats of the neural networks have been created to enable the performance of certain functions such as document translation or photos categorization to these networks.

4. Abundance of Metadata

The availability of such data is practically a revolution and includes the following types:

- Streaming data from IoT devices;
- Textual data from social media;
- Professional notes like physicians' notes and investigative transcriptions.

These data sources constitute a solid basis for constructing and deep-layered neural networks.

1.1.5 How Deep Learning Works

Deep learning is a subset of artificial intelligence that has significantly transformed the field of data analytics by enabling machines to automatically learn patterns and representations from large volumes of data. Unlike traditional programming approaches, where a system is explicitly instructed with predefined rules to solve a problem, deep learning follows a different paradigm. In this approach, the system is trained using data so that it can independently learn how to solve a problem without being manually programmed for every possible scenario. The core idea is to allow the model to discover complex patterns, relationships, and features directly from raw input data through multiple layers of neural networks.

Deep learning models are typically built using artificial neural networks consisting of multiple interconnected layers, including input layers, hidden layers, and output layers. Each layer processes the input data and passes the transformed information to the next layer. During training, the model adjusts its internal parameters (weights and biases) using optimization techniques such as backpropagation and gradient descent. This iterative learning process minimizes the error between predicted and actual outputs, enabling the model to improve its performance over time. As a result, deep learning systems are highly effective in handling complex tasks such as image recognition, speech processing, and natural language understanding, including applications like phishing detection.

Analytical Methodology in Deep Learning

In traditional analytics, the steps involved in the approach tend to include the following:

1. **Feature Engineering:** Taking out a variable (features) or picking out variables from the data set and recreating ones that can adequately describe the issue.
2. **Model Development:** All the phases of the study until a particular model is completed and picked on.
3. **Parameter Fitting:** Creating the best model that represents the data by changing the parameters of the model.

Although this approach can yield satisfactory predictive systems, it has weaknesses. The performance of the system is highly influenced by the quality of the features and the model. For example, while constructing a model for detecting fraud, one would say:

- You start with parameters that are already set and apply data manipulations in order to generate a model.
- It follows that this process may produce several thousands of variables, which will involve painstaking work to sift through this mass and select the significant ones.
- Inserting new information requires the application of the entire feature engineering exercise all over again.

CHAPTER 2

Literature Survey

1. Goel and Jain (2017) – Smishing-Classifier Model

- **Focus:** This study focused on detecting smishing attacks in mobile environments using a structured rule-based classification approach. The primary aim was to identify fraudulent SMS messages by analyzing their content and sender information.
- **Method:** The authors implemented a classification framework combining the Naïve Bayes algorithm with keyword-based feature extraction techniques. The model relied on predefined rules, including detection of suspicious keywords, URLs, and abnormal sender identities.
- **Application:** The system was capable of identifying smishing messages by validating URLs embedded in SMS content and verifying the authenticity of the sender. It helped in filtering malicious messages before reaching end users.
- **Limitation:** The model heavily depended on manually crafted rules, making it less flexible and difficult to adapt to new or evolving smishing techniques, especially those using obfuscation or new patterns.

2. Sonowal and Kuppusamy (2018) – SmiDCA Model

- **Focus:** This research introduced a machine learning-based approach for detecting smishing attacks using a correlation-driven analysis method.
- **Method:** The model extracted 39 different features from SMS messages, including linguistic and structural attributes. These features were then processed using a Random Forest classifier to improve prediction accuracy.
- **Application:** The system achieved an accuracy of approximately 96.4% in detecting fraudulent SMS messages, demonstrating the effectiveness of supervised learning in smishing detection.

- **Limitation:** The model requires continuous updates and retraining with new datasets to remain effective, as smishing techniques evolve over time.

3. Joo et al. (2017) – S-Detector Model

- **Focus:** This study aimed to enhance mobile device security by developing a smishing detection system capable of monitoring incoming SMS messages in real time.
- **Method:** The proposed system used an SMS monitoring mechanism combined with a Naïve Bayes classifier to analyze message content and detect suspicious activities.
- **Application:** The model was particularly effective in identifying smishing messages that attempted to download malicious or executable files through embedded links.
- **Limitation:** The approach had limited effectiveness in multilingual environments, making it less suitable for global applications involving diverse languages.

4. Jain and Gupta (2018) – Rule-Based Smishing Detection

- **Focus:** This work proposed a rule-based framework for detecting smishing messages using predefined classification rules.
- **Method:** The model utilized nine heuristic-based rules along with classification techniques such as PRISM to identify malicious SMS messages.
- **Application:** It achieved a high true negative rate of over 98%, effectively distinguishing legitimate messages from smishing attempts.
- **Limitation:** The system struggled to detect advanced smishing techniques that use obfuscation or bypass traditional rule-based filters.

5. Mishra and Soni (2019) – Content-Based Smishing Detection

- **Focus:** This study focused on detecting smishing messages through content analysis and contextual understanding.
- **Method:** Machine learning models combined with natural language processing (NLP) techniques were used to analyze message content, keywords, and URL patterns.
- **Application:** The system classified messages based on suspicious terms and contextual indicators, helping in identifying potential smishing attempts.

- **Limitation:** The reliance on keyword-based detection resulted in high false positive rates, where legitimate messages were sometimes incorrectly flagged as malicious.

6. Ghourabi et al. (2020) – CNN-LSTM Hybrid Model

- **Focus:** This research introduced a hybrid deep learning model for detecting SMS spam and smishing messages across multiple languages.
- **Method:** The model combined Convolutional Neural Networks (CNN) for feature extraction with Long Short-Term Memory (LSTM) networks for sequence learning.
- **Application:** It demonstrated improved performance over traditional machine learning models, especially in capturing both local patterns and sequential dependencies in text.
- **Limitation:** The model requires large labeled datasets for effective training and performance optimization.

7. Ulfath et al. (2022) – Feature Extraction-Based Smishing Detection

- **Focus:** This study aimed to improve smishing detection by enhancing feature extraction techniques.
- **Method:** The authors used TF-IDF vectorization along with feature selection methods to identify the most relevant features for classification.
- **Application:** The approach improved detection accuracy by focusing on linguistic patterns and important textual features.
- **Limitation:** The model showed limited robustness against adversarial modifications, where attackers manipulate messages to evade detection.

8. Saeki et al. (2022) – Botnet-Based Smishing Dynamics

- **Focus:** This research analyzed smishing attacks from a large-scale perspective by studying botnet-driven campaigns.
- **Method:** The study investigated communication patterns within botnets to understand how smishing attacks are distributed and executed.
- **Application:** It provided valuable insights into automated smishing strategies and attack trends, helping in designing better detection systems.

- **Limitation:** The study primarily focused on large-scale attacks and did not address individual or small-scale smishing incidents.

9. Sharaff et al. (2023) – Deep Learning-Based Smishing Identification

- **Focus:** This work focused on using advanced deep learning and NLP techniques for smishing detection.
- **Method:** The model employed LSTM and transformer-based architectures to capture contextual and sequential information from SMS data.
- **Application:** It achieved improved detection accuracy by leveraging contextual understanding and sequence modeling.
- **Limitation:** The approach involves high computational complexity and may not be suitable for real-time deployment without optimization.

10. Agrawal et al. (2023) – Hybrid Machine Learning Smishing Detection

- **Focus:** This study proposed a hybrid approach combining multiple machine learning classifiers for enhanced smishing detection.
- **Method:** The model integrated Naïve Bayes, Random Forest, and Extra Trees Classifier (ETC) in an ensemble framework.
- **Application:** It achieved an accuracy of 96.9%, demonstrating the effectiveness of combining multiple classifiers.
- **Limitation:** The performance of the model decreases when dealing with rapidly evolving smishing techniques and unseen attack patterns.

CHAPTER 3

System Analysis

System analysis is a crucial phase in the development of any application, as it involves evaluating existing solutions and identifying their limitations to design an improved system. In the context of smishing detection, analyzing current approaches helps in understanding their shortcomings in handling evolving threats and complex message patterns. This analysis provides the foundation for proposing a more advanced, accurate, and adaptive solution using modern deep learning techniques.

3.1 EXISTING SYSTEM

Current smishing detection models rely on rule-based and machine learning approaches. Rule-based techniques use keyword filtering and blacklists but fail against dynamic threats. Machine learning models such as SVM and Random Forest classify messages based on pre-defined linguistic patterns but struggle with high false positive rates. These methods lack adaptability to evolving attack strategies and require continuous manual updates. Additionally, heuristic-based detection mechanisms often misclassify legitimate SMS as threats, leading to inefficiencies. Due to the increasing volume and complexity of smishing attacks, traditional models are no longer effective. A more intelligent, adaptive, and automated solution is required to enhance smishing detection accuracy.

Disadvantages of Existing system

1. High false positive rate.
2. Ineffective against evolving attacks.
3. Requires frequent manual updates.
4. Poor feature extraction efficiency.
5. Limited generalization capability.

3.2PROPOSED SYSTEM

The proposed system integrates advanced deep learning models, including Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Bidirectional LSTM (BiLSTM), to enhance the efficiency and accuracy of smishing detection. The CNN component is responsible for extracting key textual features from SMS messages, such as important keywords, patterns, and structural elements that may indicate malicious intent. Meanwhile, the LSTM and BiLSTM layers are designed to capture sequential dependencies and contextual relationships within the text, allowing the system to understand the flow and meaning of messages more effectively. The ensemble approach combines the strengths of these models, resulting in improved classification performance, reduced false positive rates, and better adaptability to evolving smishing attack patterns. By leveraging real-world smishing datasets, the model is trained to accurately differentiate between malicious and legitimate messages under various scenarios. Furthermore, the extended CNN-LSTM-Bidirectional architecture enhances both feature extraction and sequence learning capabilities, enabling the system to detect even complex and hidden smishing attempts. Overall, the proposed system provides a robust, scalable, and intelligent solution, ensuring reliable and efficient smishing detection for modern mobile communication security.

Advantages of Proposed System

1. High classification accuracy.
2. Reduced false positive rate.
3. Adaptive to new threats.
4. Efficient feature extraction.
5. Scalable for large datasets.

CHAPTER 4

Functional Requirements

Functional requirements form an important part in the sphere of software and systems engineering as it determines the conditions which must be fulfilled in order for the system or any of its components to perform their particular tasks. Explain how inputs are transformed into outputs and what the system is expected to achieve once it is completed. Analytic computations, data creation and even some operational processes would be some instances of these requirements. Functional requirements concern the system in its aspects including but not limited to the performance capabilities while non-functional requirements address specific components and capabilities within a system, such as performance, security, or reliability.

Functional requirements are typically put forth as the condition “the system must do,” whereas for non-functional requirements the statement is put in the form of “the system shall be.” These functional requirements are provided in the system design, while non-functional requirements are maintained in the systems’ architectures. Therefore, functional requirements control the application architecture, and non-functional requirements control the technical architecture.

For the cases above and in real life, functional requirements can be confirmed and further refined through the use cases where the situation is first described. Clients provide their expectations and analysts try to observe, interact and record these expectations. These use cases are instrumental in generating the functional requirements and ensuring that the right functions are performed by the system. These steps of analyses, modeling and validation help to ensure that the requirements are met with expectation of the stakeholders and are possible to be met in reality.

4. 1 SYSTEM FUNCTIONAL REQUIREMENTS

1. User Input Interface

The system must provide an intuitive and user-friendly interface that allows users to either manually enter SMS text messages or upload message datasets for smishing detection. This interface should support real-time input as well as batch processing, ensuring flexibility for different use cases.

2. Data Preprocessing Module

The system shall include an automated preprocessing module to clean and prepare the input SMS data for analysis. This process involves removing punctuation, special characters, numbers, and stop words, as well as converting all text to lowercase. Such preprocessing ensures consistency and improves the quality of input data for accurate model prediction.

3. Feature Extraction Engine

The system must implement a feature extraction mechanism to convert textual data into numerical representations. Techniques such as Bag-of-Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF) shall be used to capture important textual features and patterns necessary for effective classification.

4. Deep Learning Model Integration

The system must integrate a pre-trained deep learning model based on CNN-LSTM architecture. This model will analyze the processed SMS data by extracting features and learning sequential dependencies, enabling accurate identification of smishing messages.

5. Smishing Detection and Classification

The system shall classify incoming SMS messages into two categories: "Smish" (malicious) and "Ham" (legitimate). The classification process must achieve high accuracy while minimizing false positive and false negative rates to ensure reliable detection.

6. Result Output Interface

The system must display the classification results in a clear and understandable format. It should also provide confidence scores, such as percentage probabilities, indicating how certain the model is about its prediction.

7. Model Comparison Functionality

The system shall support comparison between the proposed CNN-LSTM model and other machine learning models such as Support Vector Machine (SVM), Random Forest (RF), LSTM, and BiLSTM. This feature helps in evaluating performance improvements and selecting the most effective model.

8. Confusion Matrix Generation

The system must generate a confusion matrix to evaluate model performance. This matrix will display key metrics such as true positives, false positives, true negatives, and false negatives, providing a clear understanding of classification results.

9. Training and Evaluation Mode

The system must support a training mode where models can be trained using labeled datasets. It should also include an evaluation mode that calculates performance metrics such as accuracy, precision, recall, and F1-score to assess the effectiveness of the model.

10. Model Update Capability

The system shall allow periodic retraining or fine-tuning of the deep learning model using new and updated datasets. This ensures that the system remains effective against evolving smishing techniques and continuously improves its detection capability.

CHAPTER 5

Non- Functional Requirements

The non-functional requirements of a system indicate the maintained quality or response time of a system or application. For instance, a user may be configured to a system and once selected the system is expected to respond within a certain time frame: 3 seconds for 10,000 users concurrent. In a brief statement, these types of requirements explain how the system is going to fulfill its operations.

Just like deadlines in real life, they are a must while working on any product or delivering services. Many would refer to these as constraints, as these do set limitations or boundaries within which the design needs to operate. One notion of philosophy in design states ‘out of sight, out of mind’ and these graphic leaders allow exactly that so the consumer focuses on the primary objectives. Without NFRs, consumers could easily become unsatisfied due to the inefficiency of a product.

5.1 REQUIREMENTS

Accuracy

The system should maintain a minimum classification accuracy of 99% to effectively detect smishing messages with low false positives.

Performance and Speed

The system must process and classify an SMS within 2 seconds to ensure real-time or near real-time detection.

Scalability

The system should be scalable to handle large volumes of SMS data for training and prediction without performance degradation.

Reliability

The system should provide consistent results and maintain high availability with minimal downtime.

Security

The system must ensure secure handling of user data, especially when processing sensitive SMS content, by following data privacy standards.

Usability

The system should have an intuitive and user-friendly interface that allows users with minimal technical knowledge to operate it easily.

Maintainability

The system must be developed in a modular fashion, allowing easy updates, debugging, and model improvements.

Portability

The system should be deployable on different platforms (e.g., web, mobile, desktop) with minimal changes in code or configuration.

Robustness

The system must be able to handle malformed or corrupted SMS input gracefully without crashing or producing incorrect outputs.

Extensibility

The system should support integration of additional features or new detection models in the future without major structural changes.

CHAPTER 6

Feasibility Study

The feasibility study succinctly addresses whether the envisaged proposal is workable by presenting an outline and cost figures for the project. This phase is very important in determining whether the system is in concurrence with the objectives of the organization and whether it is appropriate. For purposes of carrying out a feasibility study, there need to be always criteria to be satisfied, in this case, that of understanding the system requirements. It covers the following areas:

1. Economic Feasibility:

This assesses the economic worth of the system. The expenditures incurred in the engagement of such R&D activities should be able to cover the cost incurred. The system is constructed in a way that eliminates many unnecessary costs by using free alternatives and restricting tailor-made procurements.

2. Technical Feasibility:

This assesses how the system's technical needs correspond to what is available. The completion of the system development avoids many technical specifications to minimize the alteration of the system at the time of implementation.

3. Social Feasibility:

This measures a system in terms of the aptness or readiness of users to use the system. Training and tutorials build users' self familiarity with the system aiding their seamless adoption of the system in their day-to-day work processes. User encouragement to give feedback increases the chances of enhanced adoption experience.

CHAPTER 7

Requirement Specification

7.1 Hardware Requirements

- PROCESSOR - Intel Core i3/i5
- RAM - 8 GB
- HARD DISK (SSD) - 512 GB

7.2 Software Requirements

- Operating system : Windows 11/ 10
- Coding Language : PYTHON, Jupyter Notebook

CHAPTER 8

System Design

8.1 METHODOLOGY

Data Preprocessing

Before training the models, several preprocessing techniques were applied to clean and normalize the text data. The preprocessing steps included:

- **Stop-word Removal:** Common but non-informative words (e.g., “the,” “is”) were removed.
- **Stemming and Lemmatization:** Words were reduced to their root form to unify similar terms.
- **Special Character and Digit Removal:** Unnecessary symbols, numbers, and punctuation marks were eliminated to avoid noise in the dataset.

Feature Extraction

After preprocessing, the text data was converted into numerical representations using word embedding techniques. The study experimented with various embedding methods, including:

- **Bag of Words (BOW)**
- **Term Frequency-Inverse Document Frequency (TF-IDF)**
- **GloVe Embeddings**

Due to computational constraints, the TF-IDF approach was selected as the primary method for feature extraction.

Model Development

1. **CNN (Convolutional Neural Network):** CNN extracts high-level textual features from SMS messages by detecting important patterns. It optimizes feature representation, reducing noise and improving classification accuracy.
2. **LSTM (Long Short-Term Memory):** LSTM processes sequential dependencies in text data, capturing long-term contextual relationships to enhance prediction accuracy.
3. **CNN + LSTM Ensemble:** The CNN layer extracts optimized features, which are then passed to LSTM for sequential learning. This combined approach reduces false positives and improves smishing detection accuracy.
4. **CNN + LSTM + BiLSTM Extension:** This extended model integrates a bidirectional LSTM to improve contextual understanding by processing text in both forward and backward directions, further increasing accuracy.

Results showed that the proposed CNN + LSTM model achieved **98.83% accuracy**, while the extended CNN + LSTM + BiLSTM model further improved accuracy to **98.96%**, outperforming conventional models.

Classifying Test Data

The trained model was evaluated using test data to classify SMS messages as **HAM (legitimate) or Smishing (malicious)**. The test dataset was preprocessed using the same cleaning and feature extraction techniques as the training data. The **CNN + LSTM + BiLSTM** ensemble model was applied to predict labels. The classification results were analyzed using performance metrics such as **accuracy, precision, recall, and F1-score**. The confusion matrix and ROC curve were used to validate the model's predictive performance, confirming its effectiveness in distinguishing between genuine and smishing SMS.

8.2 DATASET:

The dataset used in this research is obtained from two publicly available sources: Kaggle and Mendeley. These datasets contain SMS messages labeled as HAM (legitimate), SPAM, and SMISHING. To enhance model robustness, both datasets were combined to increase the sample size and ensure better generalization.

8.3 SYSTEM ARCHITECTURE

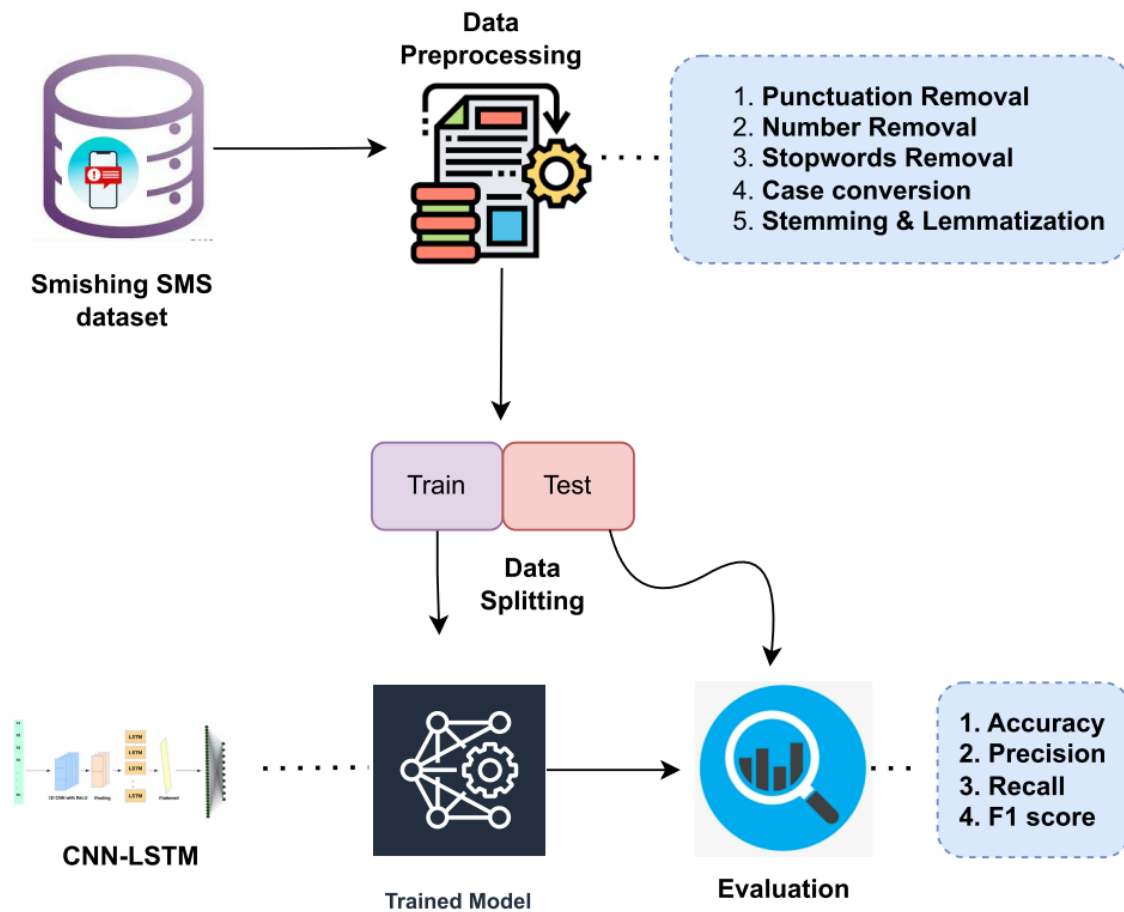


Figure 1 Architecture

CHAPTER 9

UML Diagram

UML is a model while UML is a modeling language. Unified Modeling Language is an internationally accepted modeling palette which makes it easy to visually express plans, implemented ideas and facts in terms of a system- OO. It is a language, so it can be spoken and written. It is a standardized language and there is no national ownership.

UML comprises of two basic aspects: a Meta-model and Notation. This has further enabled the specification, visualization, construction and documentation of software artifacts. It moreover fosters business modeling and other non-software systems, placing emphasis graphical notations for better communication of designs of software systems.

UML adopts the best engineering practices which have been found useful in the modeling of complex and large systems. It is a must in object-oriented software engineering for the entire process to be clear, consistent and understandable.

Goals of UML

The primary objectives of UML include:

1. Enhancing the ability to create and communicate meaningful models by providing an appropriate visual modeling language.
2. Introducing provisions for the extendibility and specialization of the basic concepts.
3. Guaranteeing freedom with respect to particular programming languages and specific development methodologies.
4. Providing a helpful underpinning to make clear the language.
5. Facilitating the development of the market of the object-oriented tools.
6. Making it possible to use more sophisticated development concepts such as collaborations, frameworks, patterns and components.

9.1 USE CASE DIAGRAM

A use case diagram in Unified Modeling Language (UML) belongs into class of behavioral diagrams and is obtained through use-case analysis. It graphically represents the functionality of the system by depicting the interactions of the actors towards the achievement of their goals which are modeled as use cases. And, it provides an overview of the dependencies between these use cases.

The main objective of use case diagrams is to indicate what tasks the system performs for each of its actors, so that, the tasks and responsibilities of the actors in the system are better understood.

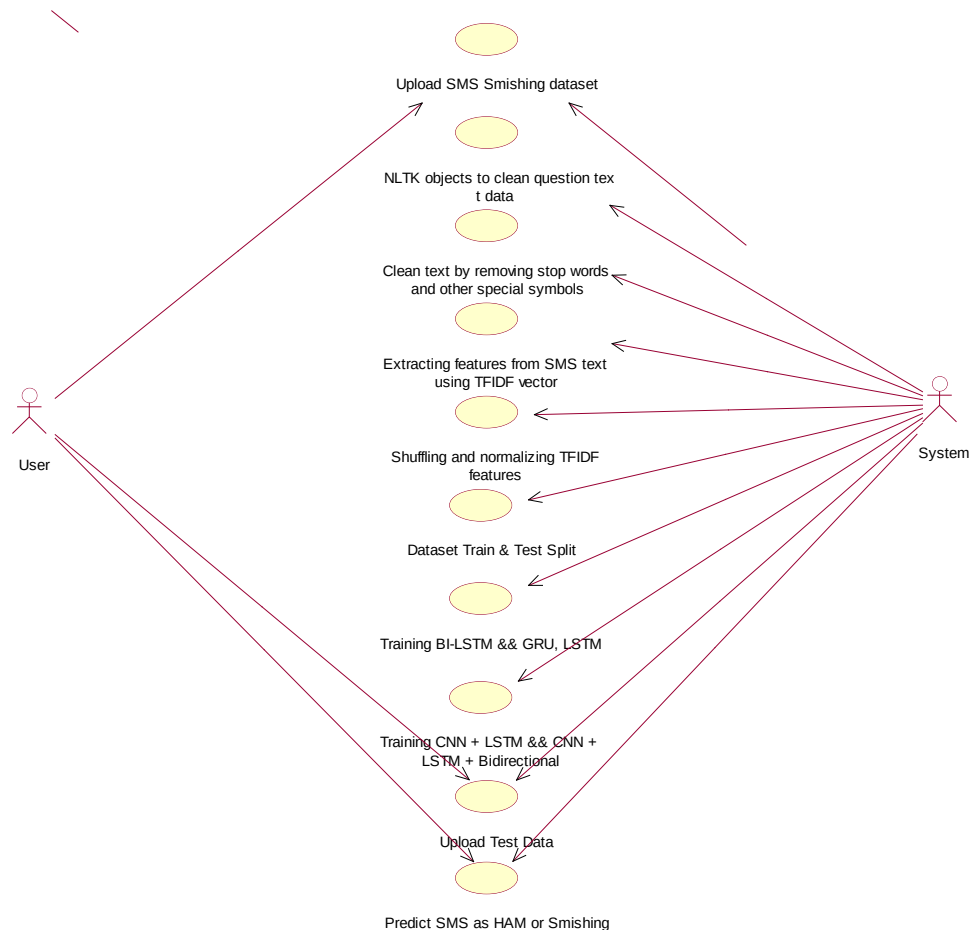


Figure 2 Use case Diagram

9.2 CLASS DIAGRAM

As it belongs to specific types within the Unified Modeling Language (UML), a class diagram is categorized as a static structure diagram in computer software construction. This is a system modeling method that models a particular system by displaying its classes (or the sets of objects), their features, methods (or functions), and their inter-relationships.

Class diagrams portray the containment and interaction of classes and provide no ambiguity about the system's blueprint. This is why they are a valuable resource for the analysis and design of object-oriented software systems.

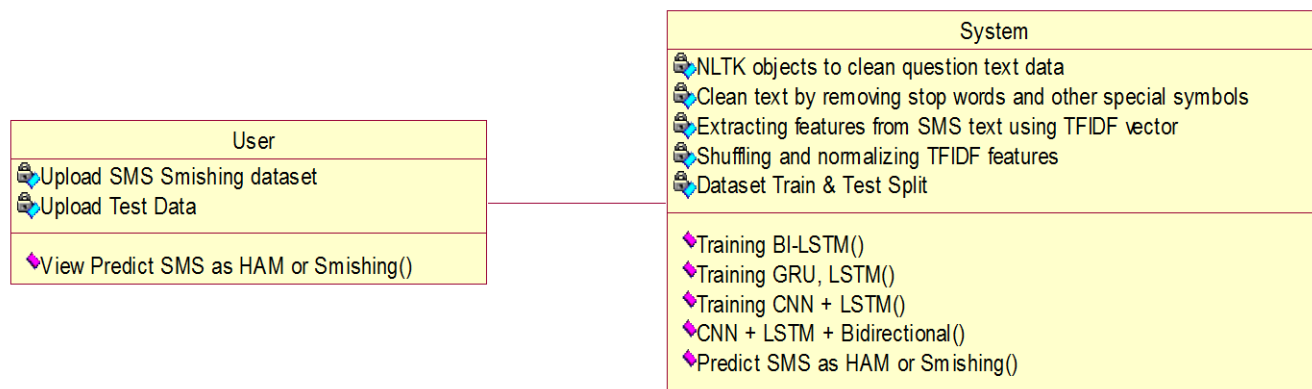


Figure 3 Class Diagram

9.3 SEQUENCE DIAGRAM

UML Sequence diagram belongs to a family of interaction diagrams – dynamic models, and documentation as well – that demonstrate how different processes work, connect and in which order they do so.

Sequence or an event diagram, it is in most cases called an event scenario or timing diagram. It, therefore, provides an elaborated view of the chronological order of events, how objects cooperated in functions over a given period in order to realize a particular process, or in other words, how the objects of the given system behaved during the realized process. These diagrams are great for depicting the behavior of the system and the subtleties of its interactions.

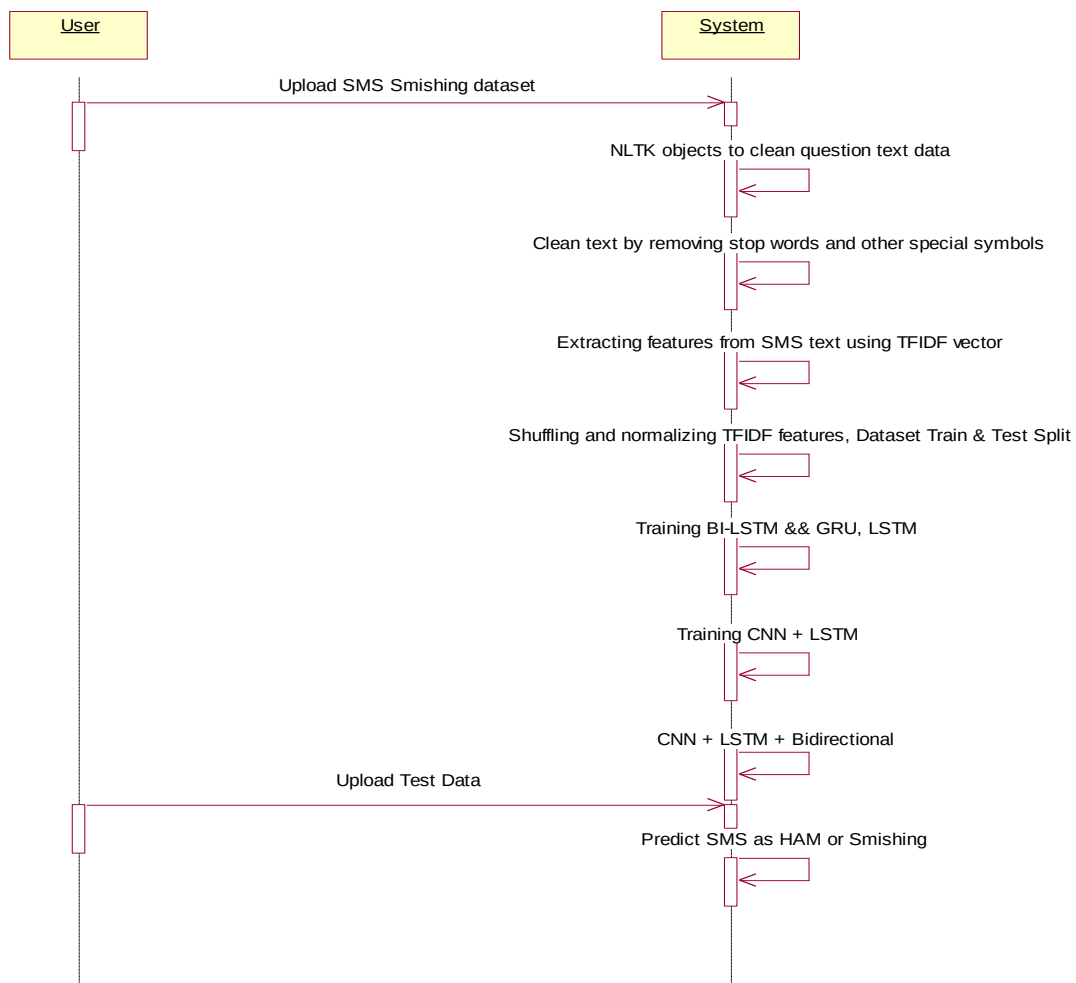


Figure 4 Sequence Diagram

9.4 COLLABORATION DIAGRAM

A collaboration diagram UML is developed to show the links and communication between objects in the system. As opposed to sequence diagrams that chronologically document the sending of messages, collaboration diagrams concentrate on the design and construction of the objects which are the center of the communication at the time of its development.

System objects are endowed with different characteristics, and many of them interrelate with one another and form relationships. The collaboration diagram or the communication diagram depicts these relations graphically in order to give information about the object structure of the system. It

assists in appreciating the reason for the existence of some objects and their mode of integration with others in the performance of certain defined functions.

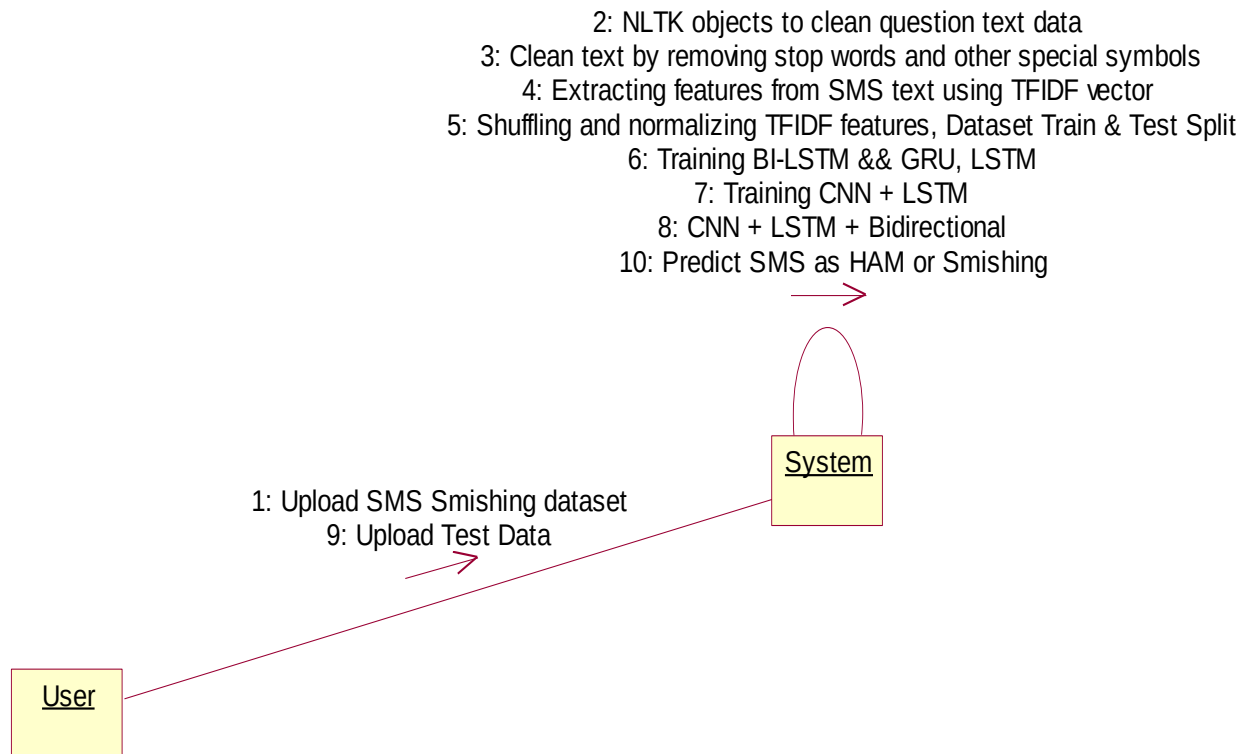


Figure 5 Collaboration Diagram

9.5 ACTIVITY DIAGRAM:

The activity diagram also contains cases of the communication and interactions of objects for the functional purpose. They include a variety of cases such as events and actions in terms of how they flow as well as how they reach the various systems edges or items.

UML activity diagrams outline the business requirements of a process from the user perspective in high level details and also the operational processes of the system explaining how different functional units operate in the context of sequential stages. They are a logically inseparable portion of each process model considering the order of operations and making decisions at each logical point associated with movement of information.

An activity diagram is a fundamental tool in both systems modeling and business processes modeling since it allows any operation to be tracked in a sequential visual representation of its flow.

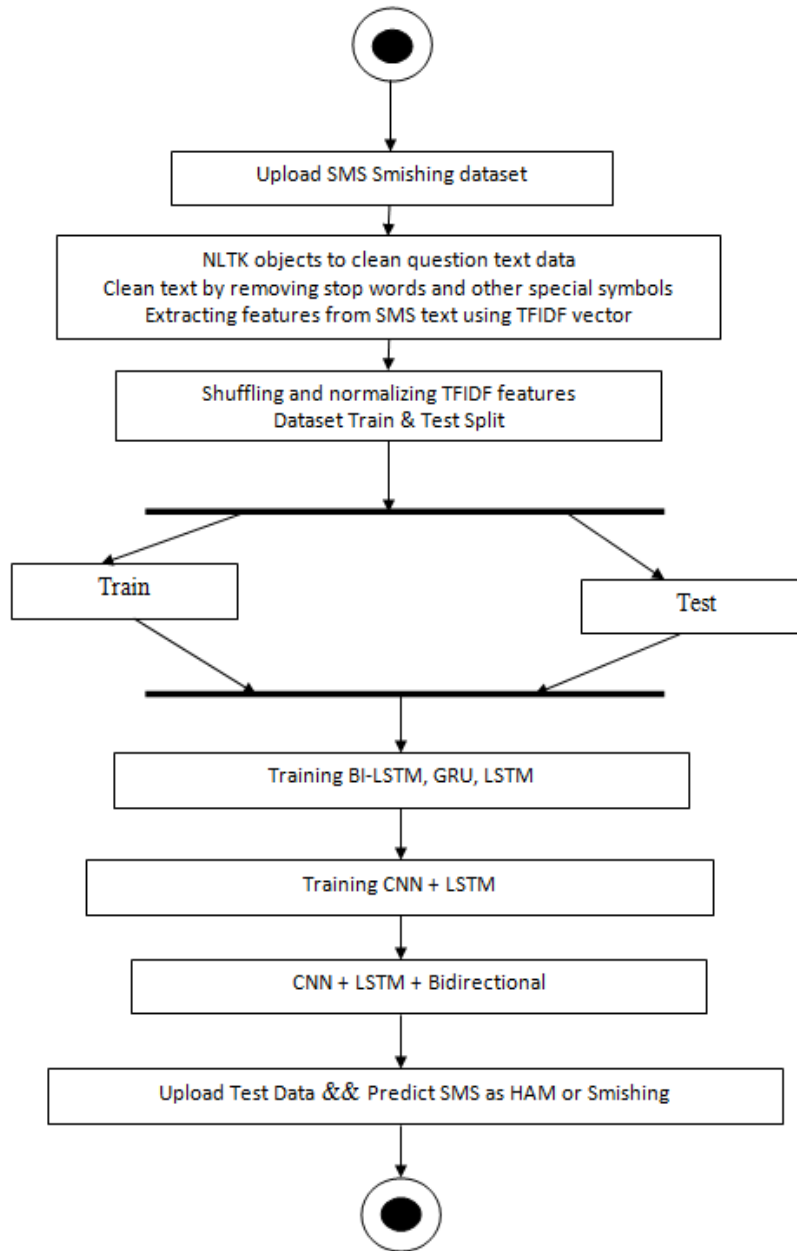


Figure 6 Activity Diagram

CHAPTER 10

Technology Description

Python is general purpose high level dynamic interpreted programming language that is used for many applications. Python programming requires a different set of skills since it supports the object-oriented programming approach. Python is clear, straightforward, and offers a number of high-level data structures making it a preferred language for programmers.

The syntax of the Python language is easy to understand. Its dynamic typing combined with the interpreted nature allows quick jump in application development and scripting. It provides support for multiple paradigms including object oriented, imperative, functional, as well as procedural programming styles to accommodate different project requirements.

Even if Python has not been targeted for a specific area like web programming, it is possible to utilize its multi-dimensional scope in web design, company solutions, implementing 3D CAD, data analysis, and other purposes. One interesting characteristic of the Python programming language is the fact that it is dynamically typed meaning that variables do not need to be explicitly declared. For example, it is possible to do `a = 10` directly.

Python proves beneficial to the user in more than one way - it did away with the need for a compilation step which in turn minimized the amount of time required for debugging and testing. Due to the efficiency of the edit-test-debug cycle, Python makes the ideal language for prototyping and tweaking designs.

10.1 PYTHON HISTORY

Python Beginning The software was conceived in the second half of the 80s. Its inception can be credited to Guido Van Rossum at Centrum Wiskunde&Informatica (CWI) in the Netherlands. Python was first hit in December 1989 and the first version (0.9.0) was released by van Rossum about February 1991.

When Python 1.0 came out in 1994, lambda, map, filter, and reduce were introduced. And in the later versions of Python, like 2.0, list comprehensions and the garbage collection system were incorporated.

In order to rectify major bugs in the language, 3.0 ok, in other words Py3K, was launched on the third day of the last month of the year of 2008. The languages that influenced the design of Python are ABC, which was an exception-handling programming language that interfaced with the Amoeba Operating System, and Modula-3.

Python has an extremely bright future due to the fact that it encompasses a lot of what its predecessors had while developing into one of the most used programming languages in the whole world.

10.2 PYTHON FEATURES

Python Characteristics Easy to learn and easy to use:

Being beginner oriented and developer 100 percent focused, python is relatively easy to learn and use.

Expressive Language:

The coding language of Python is declared as highly expressive as it is claimed that it is easy to read and to write.

Interpreted Language:

The reason for this is because Python makes use of an interpreter which takes in the line of code and executes it although it may bring out a few challenges in the debugging of codes.

Cross-platform Compatibility:

This goes to show that the system has the capacity of being used in different kinds of operating systems such as Windows, Linux, Unix and mac operating systems.

Free and Open Source:

Any person who wishes to avail Python can find it free for downloading and It is easy to acquire and modify the source code.

Object-Oriented Programming:

Object-oriented programming is achievable using Python as it provides support for classes and objects among other implementations.

Extensible:

It becomes possible to extend various functionalities and integrate performance improvements by using C/C++ languages with Python environments.

Large Standard Library:

If many different types of modules and functions are needed to develop applications quickly, then Python standard library can be a good solution.

GUI Programming Support:

It is possible to use Python to program graphical user interfaces.

Integrated with Other Languages:

The compatibility and flexibility of the programming language Python is higher as it can work with other languages particularly C, C++, Java languages.

10.3 PYTHON APPLICATIONS

Since Python is a multipurpose language, it can be used in almost all spheres of software development. Other notable fields where Python has its application include:

1. web Applications:

With its libraries for internet protocols as HTML, XML, JSON, and many others, and frameworks such as Django, Flask, Pyramid, Python is among the most popular languages for web development. Such include PythonWikiEngines, Pocoo and others.

2. Desktop GUI Applications:

Desktops, GUI can be developed using Tkinter, wxWidgets, Kivy and PyQt. For multi touch applications across platforms Kivy is good for such purpose.

3. Software Development:

Programming in Python is widely used in software development as an auxiliary language for such activities as build control, testing, management, etc.

4. Scientific and Numeric Computing:

Libraries like **SciPy**, **Pandas**, and **IPython** are popular for scientific computing, engineering, and mathematical analysis.

5. Business Applications:

Python is used for creating ERP and e-commerce solutions such as **Tryton**.

6. Console-Based Applications:

Tools like **IPython** are developed using Python for interactive computing.

7. Audio/Video Applications:

Python can develop multimedia apps like TimPlayer or cplay.

8. 3D CAD Applications:

Tools like **Fandango** are used to create CAD applications.

10.4 WHY PYTHON

Python is a versatile, user-friendly, and powerful programming language widely used across industries. Here's why Python is so popular:

- **Cross-Platform Compatibility:** Python runs on various platforms, including Windows, Mac, Linux, and Raspberry Pi, making it highly portable.
- **Simple and Readable Syntax:** Python's syntax is simple, clean, and similar to the English language, making it easier for developers to learn and write code.
- **Less Code, More Functionality:** Python allows developers to accomplish tasks with fewer lines of code compared to other programming languages.
- **Interpreter-Based Language:** Python runs on an interpreter, meaning that code can be executed immediately after being written. This makes prototyping faster and more efficient.
- **Multiple Programming Paradigms:** Python supports procedural, object-oriented, and functional programming, offering flexibility to developers.

Good to Know:

The latest major version is **Python 3**, which is used in this tutorial. Although **Python 2** is no longer updated (except for security fixes), it remains popular. You can write Python using text editors or Integrated Development Environments (IDEs) like Thonny, PyCharm, NetBeans, or Eclipse for better file management.

Python Syntax:

Python was designed for readability, using new lines instead of semicolons and indentation (whitespace) to define scope. This differs from other languages that use semicolons or curly braces.

10.5 PYTHON INSTALLATION

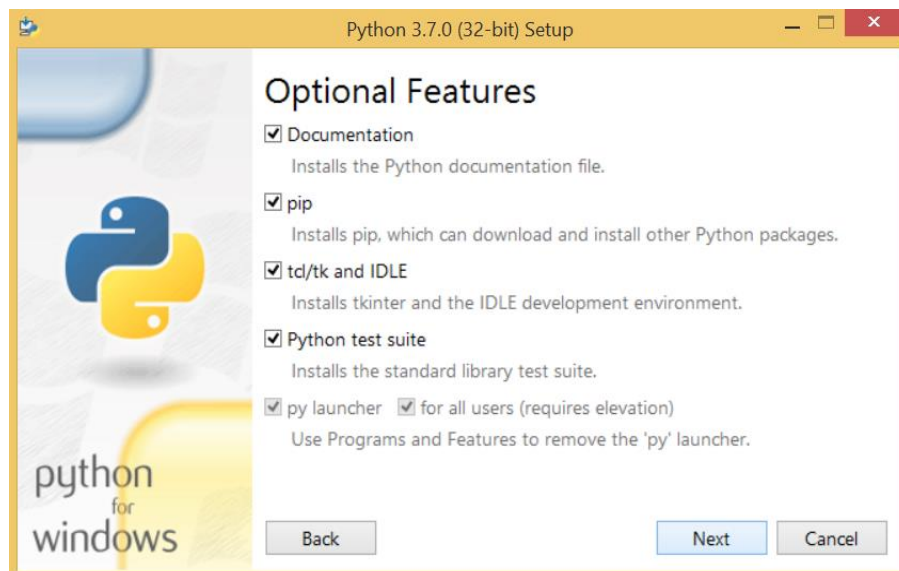
Run the Installer:

To get started, double click the Python executable file that you installed. This will also bring up the Installation Wizard. If you would like to go forward, please select Customize installation



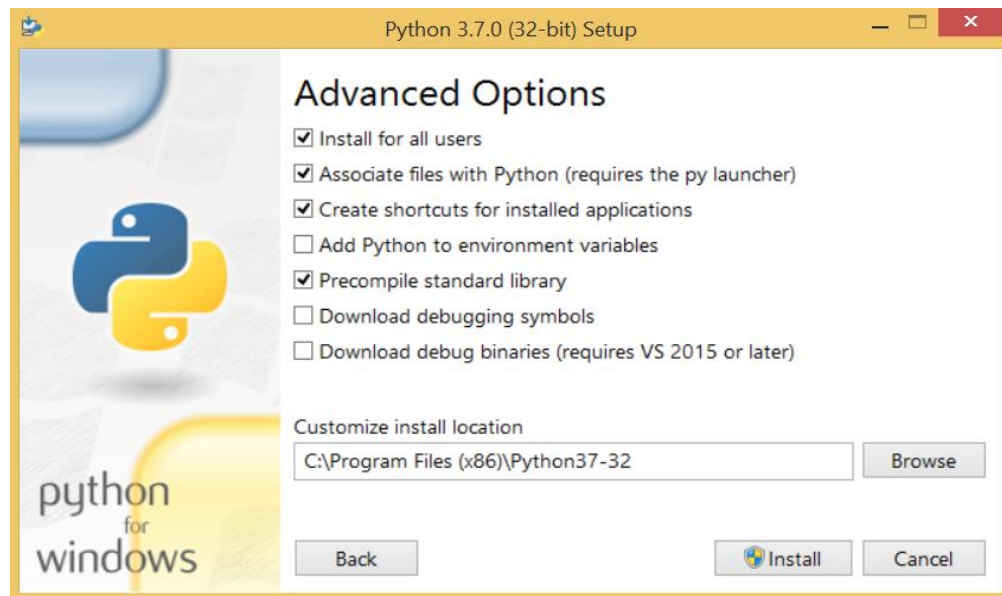
Select Optional Features:

The installation will offer additional components to augment the functionality of the software that has been brought over from the preceding window. These features come pre-activated so to proceed simply click Next.



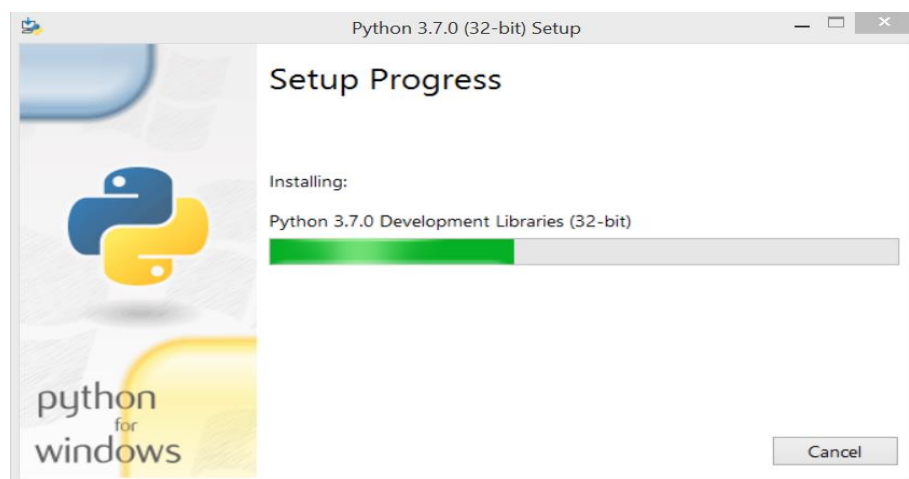
Advanced Options:

The following window will show advanced options. These options can be installed according to your requirements though it may take a relatively longer time. To proceed, tick the first box 'Install for all users' and click Next.



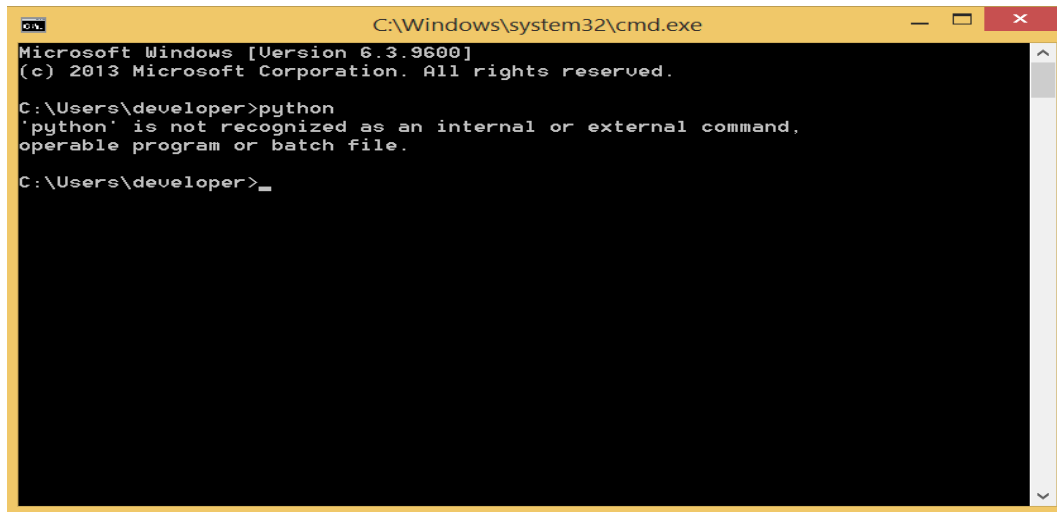
Start the Installation:

Now that you are in the installation window, click on the Install button in order to start the installation process. You can now start the installation of Python 3.6.6.



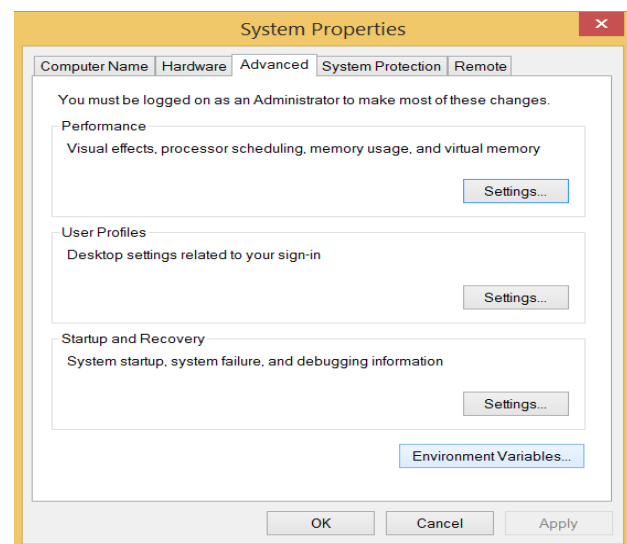
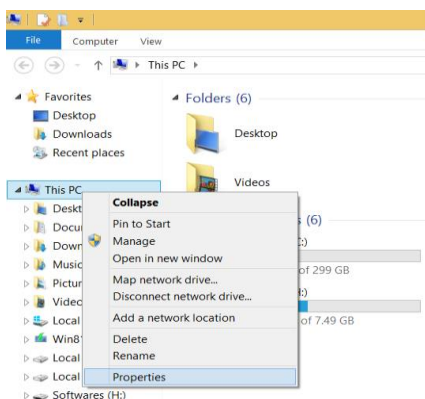
Run Python:

To begin the compilation on Python 2 please open the Command Prompt and write python or python3 for Python 3. In case of an error, you need to set the path.

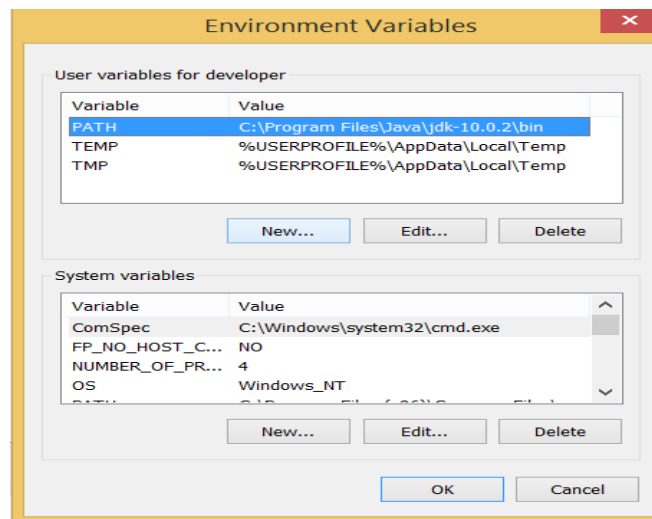


Run Python:

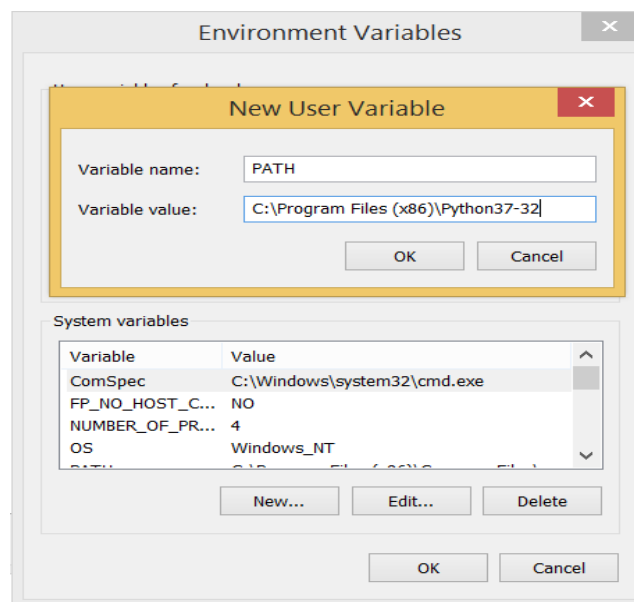
To begin the compilation on Python 2 please open the Command Prompt and write python or python3 for Python 3. In case of an error, you need to set the path.



Add the new path variable in the user variable section.



Specify the directory of the installation as per the image given below and add it as PATH in the new variables user section.



Restart the Command Prompt:

Start by opening the CMD again then type python. This time around, the python interpreter shell will initiate and from here it will be possible to issue python commands.

CHAPTER 11

Sample Code

```
1 #loading python classes and packages
2 import pandas as pd
3 import numpy as np
4 from string import punctuation
5 from nltk.corpus import stopwords
6 import nltk
7 from nltk.stem import WordNetLemmatizer
8 import pickle
9 from nltk.stem import PorterStemmer
10 import os
11 from sklearn.preprocessing import StandardScaler
12 from sklearn.model_selection import train_test_split
13 from sklearn.feature_extraction.text import TfidfVectorizer #loading tfidf
14 vector
15 from sklearn.metrics import accuracy_score
16 from keras.callbacks import ModelCheckpoint
17 from keras.layers import Bidirectional, GRU, LSTM, Conv1D, MaxPooling1D,
18 RepeatVector, GlobalAveragePooling1D#loading GRU, bidirectional, lstm and CNN
19 from sklearn.model_selection import train_test_split
20 from keras.layers import Dense, Dropout, Activation, Flatten
21 from keras.models import Sequential, Model, load_model
22 from keras.utils.np_utils import to_categorical
23 import matplotlib.pyplot as plt
24 from sklearn.metrics import precision_score
25 from sklearn.metrics import recall_score
26 from sklearn.metrics import f1_score
27 from sklearn.metrics import confusion_matrix
28 import seaborn as sns
29 import matplotlib.pyplot as plt
30 from sklearn.metrics import roc_curve
31 from sklearn.metrics import roc_auc_score
32 from sklearn import metrics
33 import warnings
34 #defining NLTK objects to clean question text data
35 #define object to remove stop words and other text processing
36 stop_words = set(stopwords.words('english'))
37 lemmatizer = WordNetLemmatizer()
38 ps = PorterStemmer()
39 In [3]:
40 #define function to clean text by removing stop words and other special
41 symbols
42 def cleanText(doc):
43     tokens = doc.split()
44     table = str.maketrans('', '', punctuation)
45     tokens = [w.translate(table) for w in tokens]
46     tokens = [word for word in tokens if word.isalpha()]
```

```

47     tokens = [w for w in tokens if not w in stop_words]
48     tokens = [word for word in tokens if len(word) > 1]
49     tokens = [ps.stem(token) for token in tokens]
50     tokens = [lemmatizer.lemmatize(token) for token in tokens]
51     tokens = ' '.join(tokens)
52     return tokens
53 In [4]:
54 #loading and displaying mendely SMS smishing dataset
55 mendely_dataset = pd.read_csv("Dataset/Dataset_5971.csv")
56 mendely_dataset
57 Out[4]:
58 #loading and displaying Kaggle SMS smishing dataset
59 kaggle_dataset = pd.read_csv("Dataset/kaggle_smishing.csv", encoding = "ISO-
60 8859-1")
61 kaggle_dataset
62 #defining global variables to add processed text and labels
63 X = []
64 Y = []
65 In [7]:
66 #defining function to loop and read each SMS and then clean and return
67 process text to extract features
68 def loadDataset(sms_text, labels):
69     for i in range(len(sms_text)):#loop each sms from given dataset sms texy
70         sms = str(sms_text[i])
71         sms = sms.strip("\n").strip().lower()#convert sms to lower and remove
72 empty text
73         if len(sms) > 0:
74             label = labels[i].strip().lower()
75             sms = cleanText(sms)#clean SMS text and add to X and Y training
76 array
77             X.append(sms)
78             if label == "ham":
79                 Y.append(0)
80             else:
81                 Y.append(1)
82 In [8]:
83 #if sms already cleaned then load them else processed and load them
84 if os.path.exists("model/X.npy"):
85     X = np.load("model/X.npy")
86     Y = np.load("model/Y.npy")
87 else:
88     #extract labels and text from both dataset and then clean them
89     sms_text = mendely_dataset['TEXT'].ravel()
90     labels = mendely_dataset['LABEL'].ravel()
91     loadDataset(sms_text, labels)#calling function to clean mendely dataset
92     sms_text = kaggle_dataset['v2'].ravel()
93     labels = kaggle_dataset['v1'].ravel()
94     loadDataset(sms_text, labels)#calling function to clean KAGGLE dataset
95     X = np.asarray(X)
96     Y = np.asarray(Y)
97     np.save("model/X", X)
98     np.save("model/Y", Y)
99 print("Dataset SMS Cleaning & Processing Completed")
100 print("Total SMS found in Dataset = "+str(X.shape[0]))
101 Dataset SMS Cleaning & Processing Completed
102 Total SMS found in Dataset = 11543
103 In [9]:

```

```
104 #visualizing graph of number of duplicate and non-duplicate (isolate)
105 questions
106 labels, count = np.unique(Y, return_counts = True)
107 height = count
108 bars = ['Ham', 'Smishing']
109 y_pos = np.arange(len(bars))
110 plt.figure(figsize = (4, 3))
111 plt.bar(y_pos, height)
112 plt.xticks(y_pos, bars)
113 plt.xlabel("Dataset Class Label Graph")
114 plt.ylabel("Count")
115 plt.xticks(rotation=90)
116 plt.show()
```

CHAPTER 12

Testing

12.1 INTRODUCTION

Software engineers draw a distinction between software fault and software failure. A fault will mean an error that is either a bug in programming or a defect in a certain program's semantic. A fault may or may not result in a failure. A failure on the other hand is such that a software has not performed as expected by the end user. Under certain conditions such as when the faulty code is run on the CPU, or even when it is migrated to another hardware machine or when it is enhanced with new capabilities, faults become failures.

Testing a software application is always essential in the process of analyzing the software with the aim of validating its quality and providing the stakeholders with relevant information. It states the errors that it contains and so guarantees that a software has a certain amount of standards that it fulfills with users and system requirements.

12.1.1 System Testing and Implementation

System testing is concerned with testing the logic of individual module codes and debugging them before their assembly into larger subsystems. This involves:

1. **Module Integration Testing:** This tests the linkages of modules to one another.
2. **System Testing:** This is done to check that the whole system conforms to the requirements provided.
3. **Acceptance Testing:** This goal is to represent the abilities of the system to the customers.

There are two methods that could be used in selecting test cases which include:

- **Black Box Testing** - Testing of the external behavior with respect to the system requirements specification.

- **White Box Testing** - Concentrates on the assessment of internal logic and aims at coverage of the code.

12.1.2 Testing Techniques

Testing discovers mistakes made in the software by executing the software under conditions that were planned in advance. Two main approaches are:

1. **Black Box Testing:**

Emphasizes functional specification and assesses external behavior without any interference on internal logic. It detects errors such as interface error, the database access error, performance as well.

2. **White Box Testing:**

Deals with the internal logic by scrutinizing module flow graphs and analyzing logical paths, loops and data structures to be tested.

12.1.3 Testing Strategies

To ensure their applications are free from errors and comply with the required functionality, the effective performance of software testing is paramount. Given below are the top primary kinds of software testing strategies used in the software development process:

Unit Testing

Unit testing tends to assure the correctness of the implementation of individual units or components of the software. It is usually conducted to ensure the correctness of internal logic, and that for a particular input element expected output is given. The unit testing is said to be structural and invasive since it depends on the knowledge of the structure of the software developed.

The main objective of developers when using unit testing is to ensure the validity of each decision branch and each internal code path. This guarantees all business logic and configurations function as intended. Unit testing is usually done at the level of components and it helps in detecting faults very early in the development cycle.

A good example would be a system with three modules: a Reputation Module, Route Discovery Module and Audit Module. All of these systems would be put through a separate test. Errors are

recognized during this stage, patches are made and the patched executable units are checked before they are incorporated.

Integration Testing

Integration testing is usually done after unit testing and it is concerned with putting several units or components together into one cohesive system as they were meant to work. It is event driven, concentrating on functional interconnections and ultimately the interaction of the components of the system.

While unit testing touches on the performance of each individual component, unit testing is aimed at confirming that all the component systems do not create problems when put together. These tests determine whether there are discrepancies in attached data and also the communication lines between modules so as to enhance unity and harmony in system execution.

System Testing

As the name states, system testing tests the final, integrated software product to ensure everything is functioning according to the requirements. It deals with the global layout and just like the end to end testing, guarantees normal results given the particular components are used in a realistic way.

One example of system testing is configuration-oriented system integration testing, which checks how processes interact at various integration points. System testing focuses on process flows and assesses whether the system is reliable and meets the required expectations.

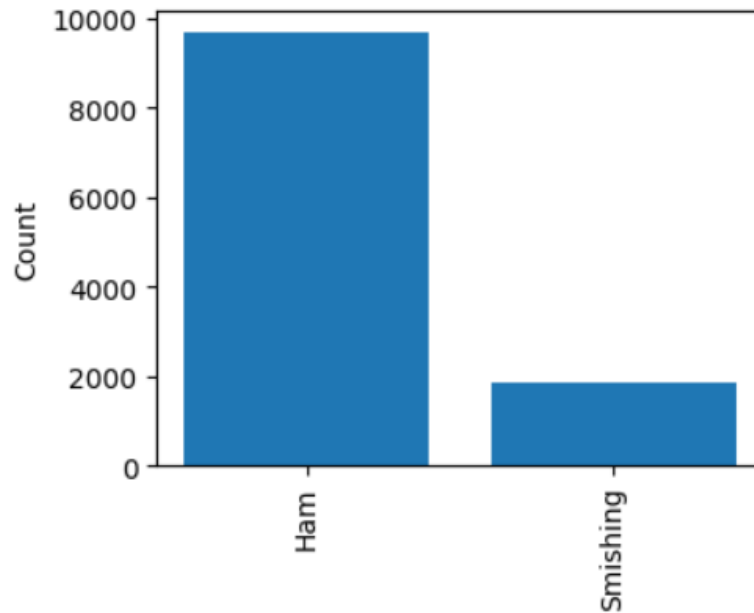
Functional Testing

Functional testing establishes whether the system implemented meets the business and technical requirements stated in system documents and users' manuals. It offers a systematic approach for verifying that all required functions have been implemented correctly and as expected by users.

Functional testing was done by using each business process or feature and user interaction against documented requirements. This helps ensure that all features work as they are intended to work without any problems.

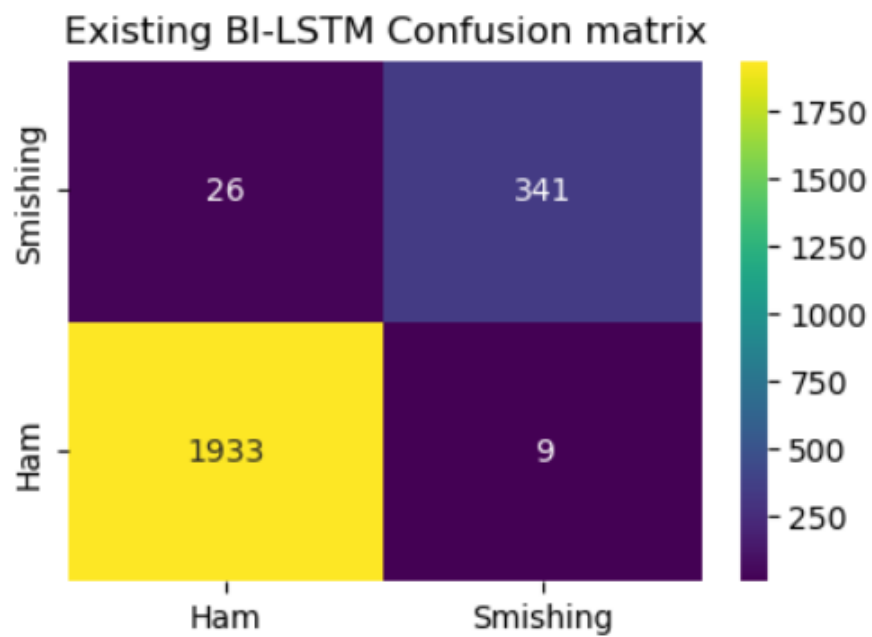
CHAPTER 13

Screenshots

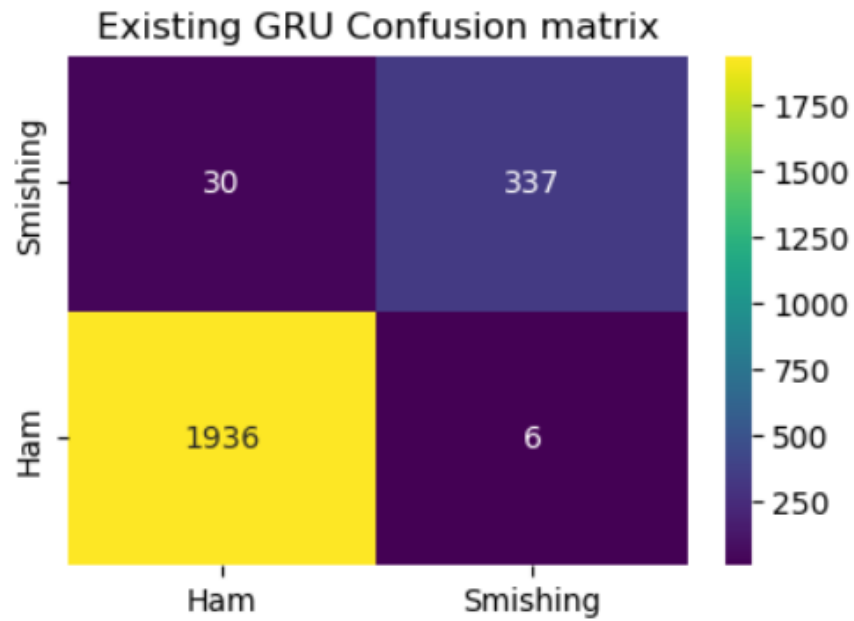


Dataset Class Label Graph

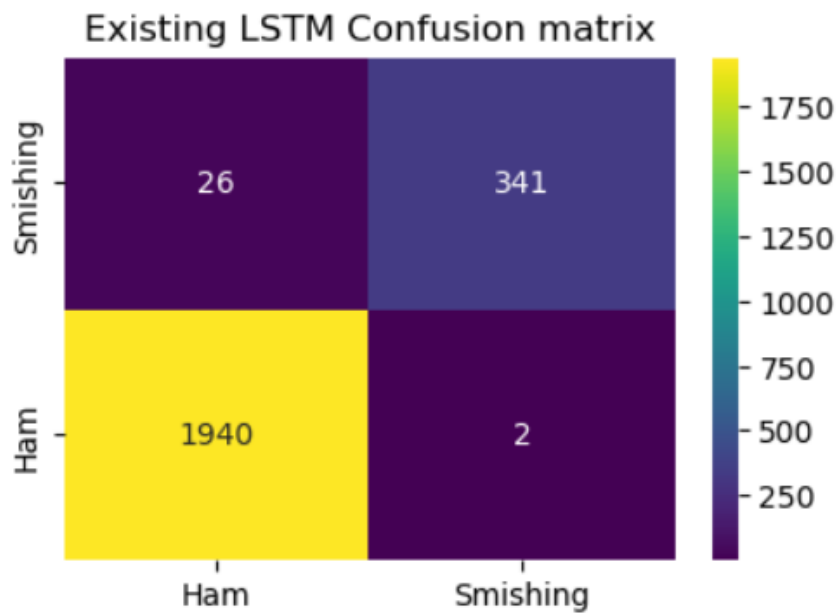
Screen 1 Dataset Class Label Graph



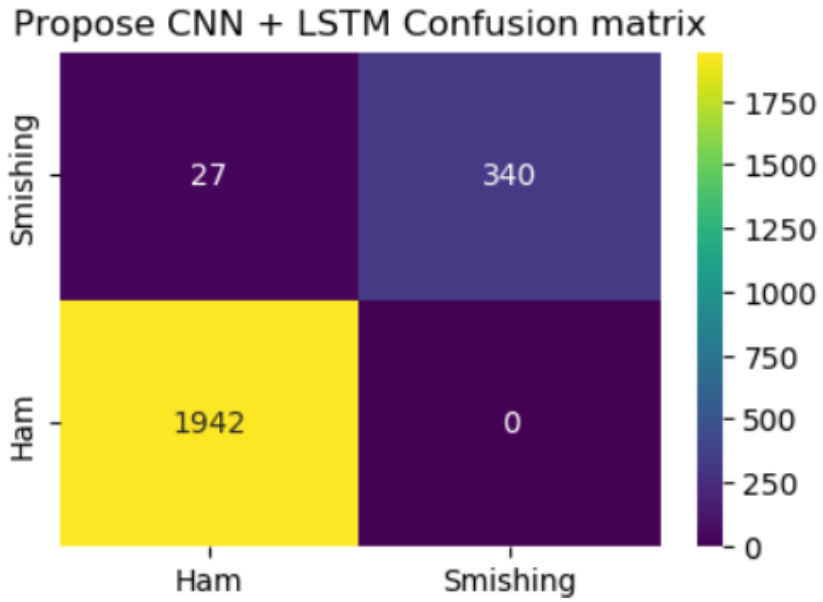
Screen 2 BI-LSTM



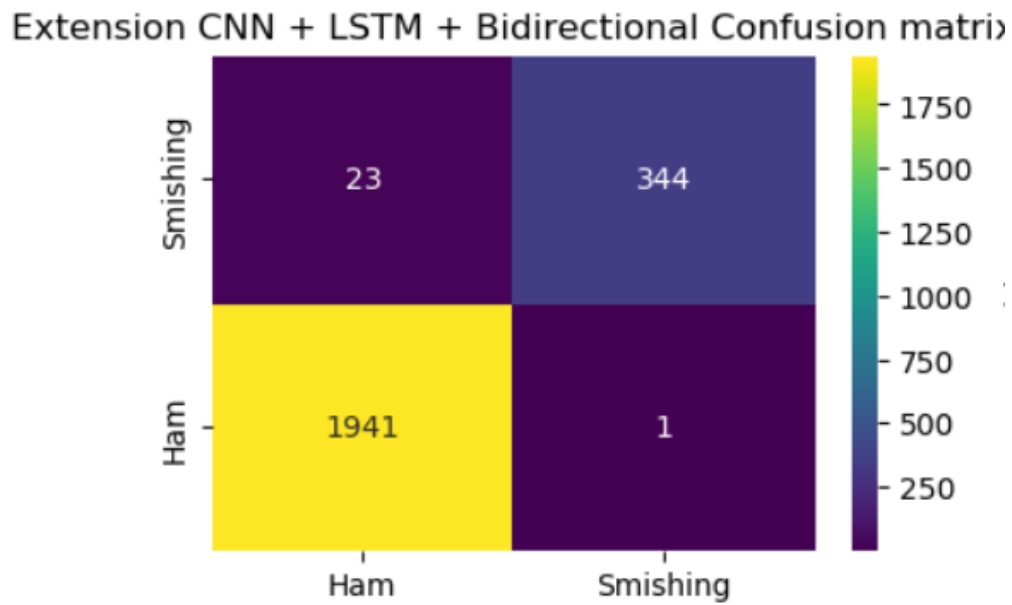
Screen 3 GRU Algorithm



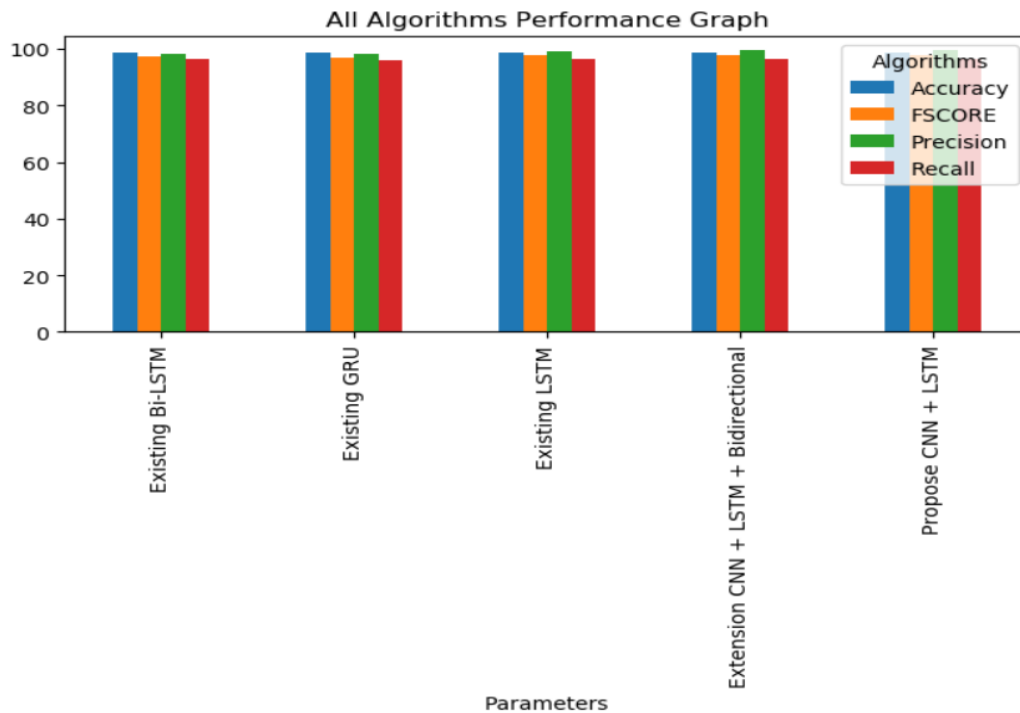
Screen 4 LSTM Algorithms



Screen 5 CNN + LSTM



Screen 6 CNN + LSTM + Bidirectional



Screen 7 All Algorithms Performance Graph

Enhancing Smishing Detection: A Deep Learning Approach for Improved Accuracy and Reduced False Positives

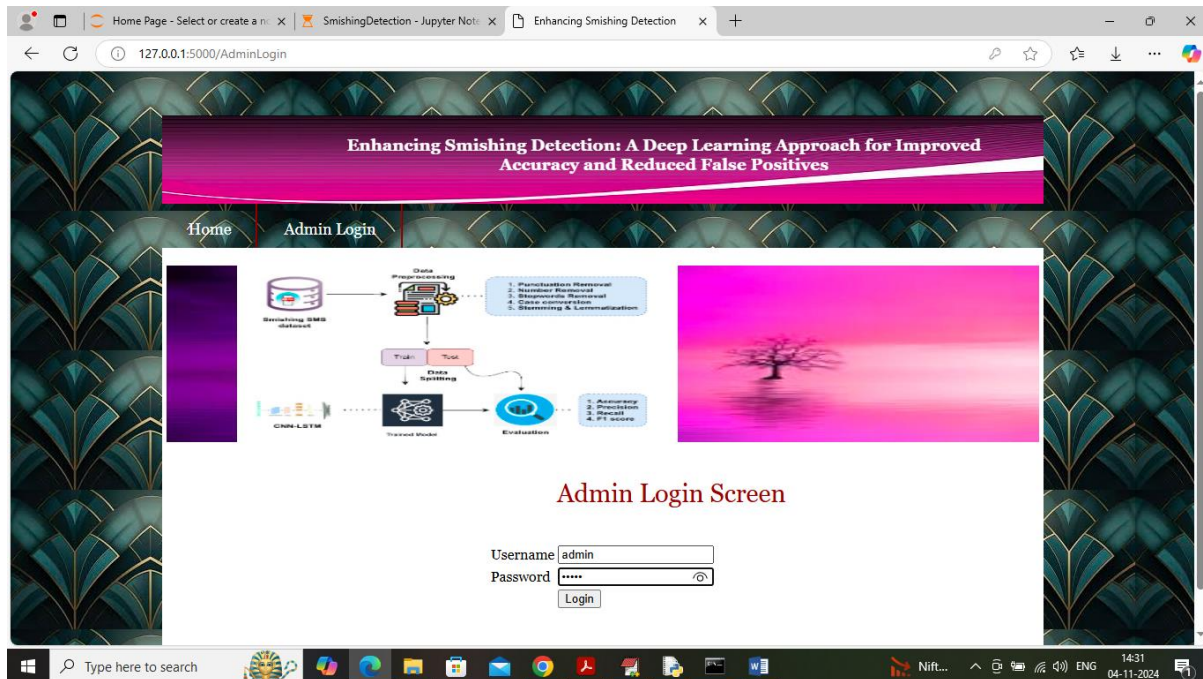
Home Admin Login

Smishing Detection Process:

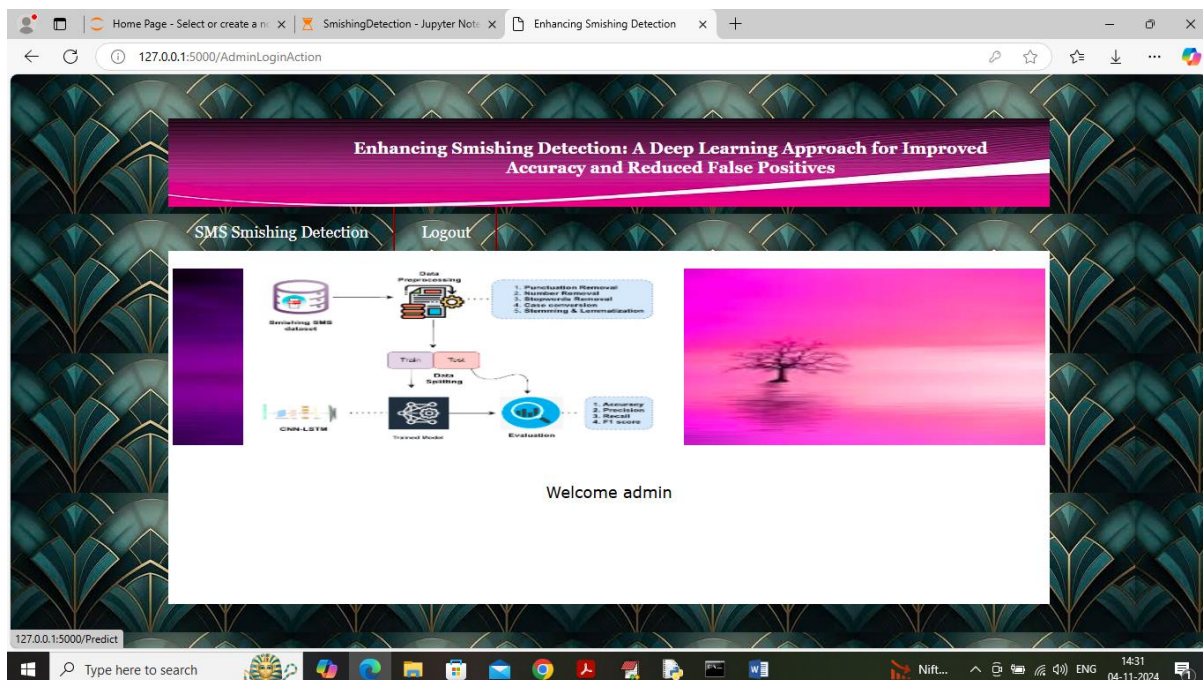
1. Preprocessing: Removal of Stopwords, Removal of Punctuation, Removal of Special Characters, Stemming & Lemmatization
2. Training: Training the CNN-LSTM model
3. Testing: Testing the model on new data
4. Evaluation: Evaluating the model's performance using Accuracy, Precision, Recall, and F1 score

The widespread use of smartphones and their constant connection to the Internet makes them vulnerable to phishing attacks. Phishing is the act of sending malicious content such as emails to unsuspecting individuals. Smishing, a hybrid of Short Message Service (SMS) and phishing is a well-known cybersecurity problem in which attackers send malicious SMS messages to their targets. This practice is deceptive and aims to mislead individuals into exposing personal information or completing certain activities through text messages. Although researchers have presented various techniques to detect smishing, there is still a lack of methods to significantly reduce false-positive predictions, which are incorrect classifications of legitimate messages as malicious.

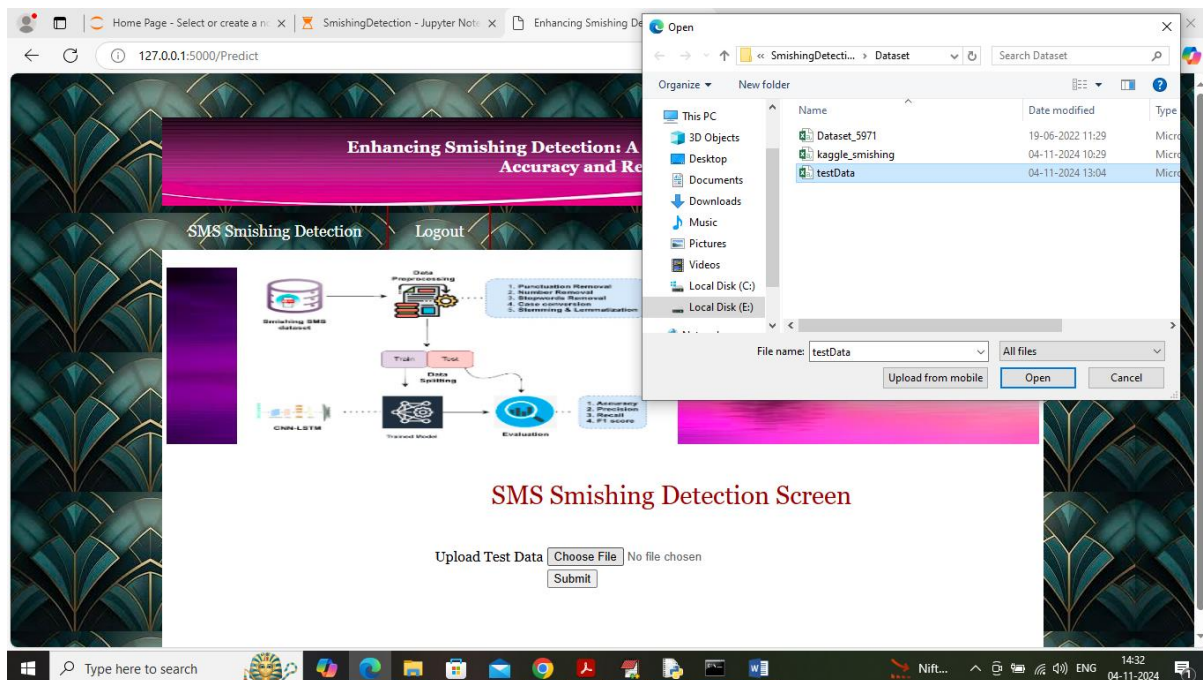
Screen 8 Home Page



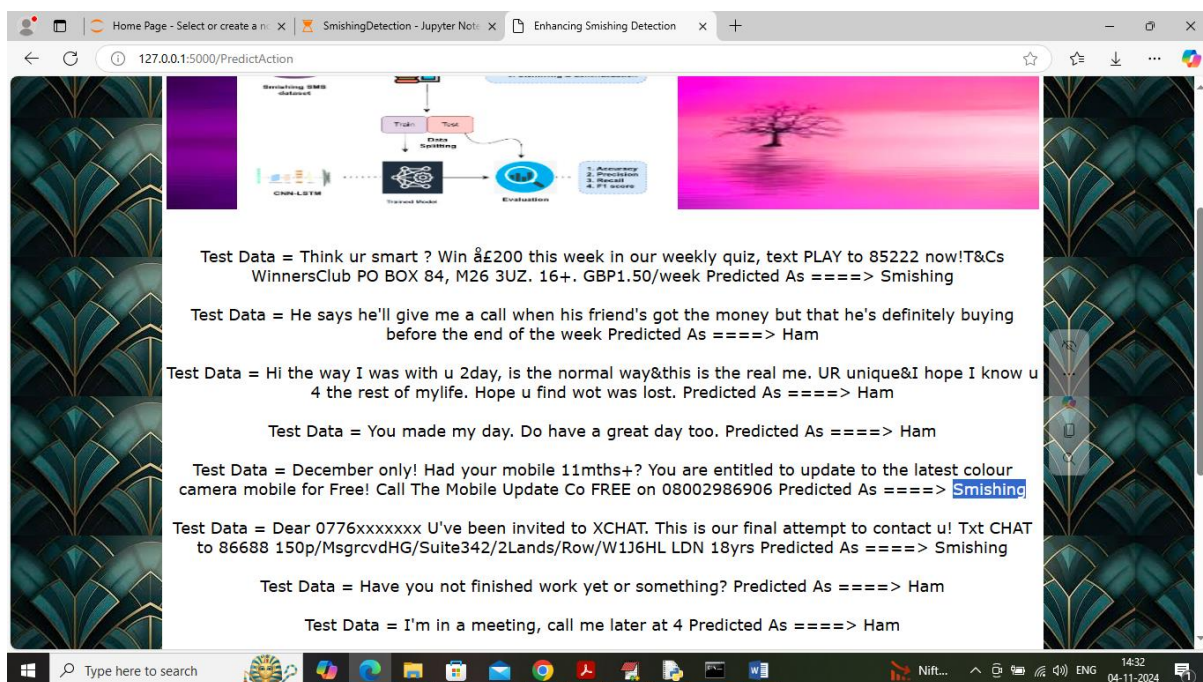
Screen 9 Admin Login



Screen 10 Admin Home



Screen 11 Upload 'testData.csv' file



Screen 12 SMS Smishing Detection

CHAPTER 14

Conclusion

This project presents an effective deep learning-based solution to detect smishing messages, aiming to reduce false positives while maintaining high accuracy. By combining CNN and LSTM into an ensemble model, the system successfully extracts meaningful features and captures contextual relationships in SMS content. The proposed model outperformed traditional machine learning and existing deep learning approaches, achieving up to 98.83% accuracy. Further enhancements using CNN + LSTM + Bidirectional improved accuracy to 98.96%. Datasets from Kaggle and Mendeley were used for robust evaluation, and preprocessing techniques ensured clean, optimized inputs. Overall, the project demonstrates how deep learning can significantly improve mobile cybersecurity by reliably detecting smishing attacks and protecting users from phishing threats.

CHAPTER 15

Bibliography

- [1] S. Bajpai and D. Radha, “Smart phone as a controlling device for smart home using speech recognition,” in *Proc. Int. Conf. Commun. Signal Process. (ICCSP)*, Apr. 2019, pp. 701–705.
- [2] R. E. Madrid, F. A. Ramallo, D. E. Barraza, and R. E. Chaile, “Smartphone-based biosensor devices for healthcare: Technologies, trends, and adoption by end-users,” *Bioengineering*, vol. 9, no. 3, p. 101, Mar. 2022.
- [3] K. K. Ibrahim and A. J. Obaid, “Fraud usage detection in internet users based on log data,” *Int. J. Nonlinear Anal. Appl.*, vol. 12, no. 2, pp. 2179–2188, 2021.
- [4] G. Sonowal and G. Sonowal, “Introduction to phishing,” in *Phishing and Communication Channels: A Guide to Identifying and Mitigating Phishing Attacks*. Springer, 2022, pp. 1–24.
- [5] G. Desolda, L. S. Ferro, A. Marrella, T. Catarci, and M. F. Costabile, “Human factors in phishing attacks: A systematic literature review,” *ACM Comput. Surv.*, vol. 54, no. 8, pp. 1–35, Nov. 2022.
- [6] G. Brown and P. M. Greenfield, “Staying connected during stay-at-home: Communication with family and friends and its association with wellbeing,” *Hum. Behav. Emerg. Technol.*, vol. 3, no. 1, pp. 147–156, 2021.
- [7] S. Mishra and D. Soni, “Smishing detector: A security model to detect smishing through SMS content analysis and URL behavior analysis,” *Future Gener. Comput. Syst.*, vol. 108, pp. 803–815, Jul. 2020.
- [8] H. A. Wahsheh and M. S. Al-Zahrani, “Lightweight cryptographic and artificial intelligence models for anti-smishing,” in *Proc. Int. Conf. Emerg. Technol. Intell. Syst.*, vol. 2. Cham, Switzerland: Springer, 2021, pp. 483–496.
- [9] G. Canova, M. Volkamer, C. Bergmann, R. Borza, B. Reinheimer, S. Stockhardt, and R. Tenberg, “Learn to spot phishing URLs with the Android NoPhish app,” in *Proc. IFIP World Conf. Inf. Secur. Educ. (WISE)*, Hamburg, Germany. Cham, Switzerland: Springer, May 2015, pp. 87–100.

- [10] A. U. Surwade, "Phishing e-mail is an increasing menace," *Int. J. Inf. Technol.*, vol. 12, no. 2, pp. 611–617, Jun. 2020.
- [11] K. Gradoń, "Crime in the time of the plague: Fake news pandemic and the challenges to law-enforcement and intelligence community," *Soc. Register*, vol. 4, no. 2, pp. 133–148, Apr. 2020.
- [12] S. J. Delany, M. Buckley, and D. Greene, "SMS spam filtering: Methods and data," *Expert Syst. Appl.*, vol. 39, no. 10, pp. 9899–9908, Aug. 2012.
- [13] A. Ghourabi, "SM-detector: A security model based on BERT to detect SMiShing messages in mobile environments," *Concurrency Comput., Pract. Exper.*, vol. 33, no. 24, p. 6452, Dec. 2021.
- [14] A. K. Jain, D. Goel, S. Agarwal, Y. Singh, and G. Bajaj, "Predicting spam messages using back propagation neural network," *Wireless Pers. Commun.*, vol. 110, no. 1, pp. 403–422, Jan. 2020.
- [15] O. Sova, O. Turinskyi, A. Shyshatskyi, V. Dudnyk, R. Zhyvotovskiy, Y. Prokopenko, T. Hurskyi, V. Hordiichuk, A. Nikitenko, and A. Remez, "Development of an algorithm to train artificial neural networks for intelligent decision support systems," *Eastern-Eur. J. Enterprise Technol.*, vol. 1, no. 9, pp. 46–55, Feb. 2020.
- [16] D. Goel and A. K. Jain, "Smishing-classifier: A novel framework for detection of smishing attack in mobile environment," in *Proc. Int. Conf. Next Gener.Comput.Technol. (NGCT)*, Dehradun, India. Cham, Switzerland: Springer, Oct. 2017, pp. 502–512.
- [17] G. Sonowal and K. S. Kuppusamy, "SmiDCA: An anti-smishing model with machine learning approach," *Comput. J.*, vol. 61, no. 8, pp. 1143–1157, Aug. 2018.
- [18] J. W. Joo, S. Y. Moon, S. Singh, and J. H. Park, "S-detector: An enhanced security model for detecting smishing attack for mobile computing," *Telecommun. Syst.*, vol. 66, no. 1, pp. 29–38, Sep. 2017.
- [19] A. K. Jain and B. B. Gupta, "Rule-based framework for detection of smishing messages in mobile environment," *Proc. Comput. Sci.*, vol. 125, pp. 617–623, Jan. 2018.
- [20] S. Mishra and D. Soni, "A content-based approach for detecting smishing in mobile environment," in *Proc. Int. Conf. Sustain.Comput. Sci., Technol. Manage. (SUSCOM)*, 2019.